



Chapter 6: Entity-Relationship Model





Chapter 6: Entity-Relationship Model

- Design Process
- Modeling
- Constraints
- E-R Diagram
- Design Issues
- Weak Entity Sets
- Extended E-R Features
- Design of the Bank Database
- Reduction to Relation Schemas
- Database Design
- UML





Modeling

- A *database* can be modeled as:
 - a collection of entities,
 - relationship among entities.
- An **entity** is an object that exists and is distinguishable from other objects.
 - Example: specific person, company, event, plant
- Entities have *attributes*
 - Example: people have *names* and *addresses*
- An **entity set** is a set of entities of the same type that share the same properties.
 - Example: set of all persons, companies, trees, holidays





Entity Sets *customer* and *loan*

customer_id customer_ customer_ customer_ loan_ amount
 name street city number

321-12-3123	Jones	Main	Harrison
019-28-3746	Smith	North	Rye
677-89-9011	Hayes	Main	Harrison
555-55-5555	Jackson	Dupont	Woodside
244-66-8800	Curry	North	Rye
963-96-3963	Williams	Nassau	Princeton
335-57-7991	Adams	Spring	Pittsfield

customer

L-17	1000
L-23	2000
L-15	1500
L-14	1500
L-19	500
L-11	900
L-16	1300

loan





Relationship Sets

- A **relationship** is an association among several entities

Example:

<u>Hayes</u>	<u>depositor</u>	<u>A-102</u>
<i>customer</i> entity	relationship set	<i>account</i> entity

- A **relationship set** is a mathematical relation among $n \geq 2$ entities, each taken from entity sets

$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

where $(e_1, e_2, \dots, e_n)$ is a relationship

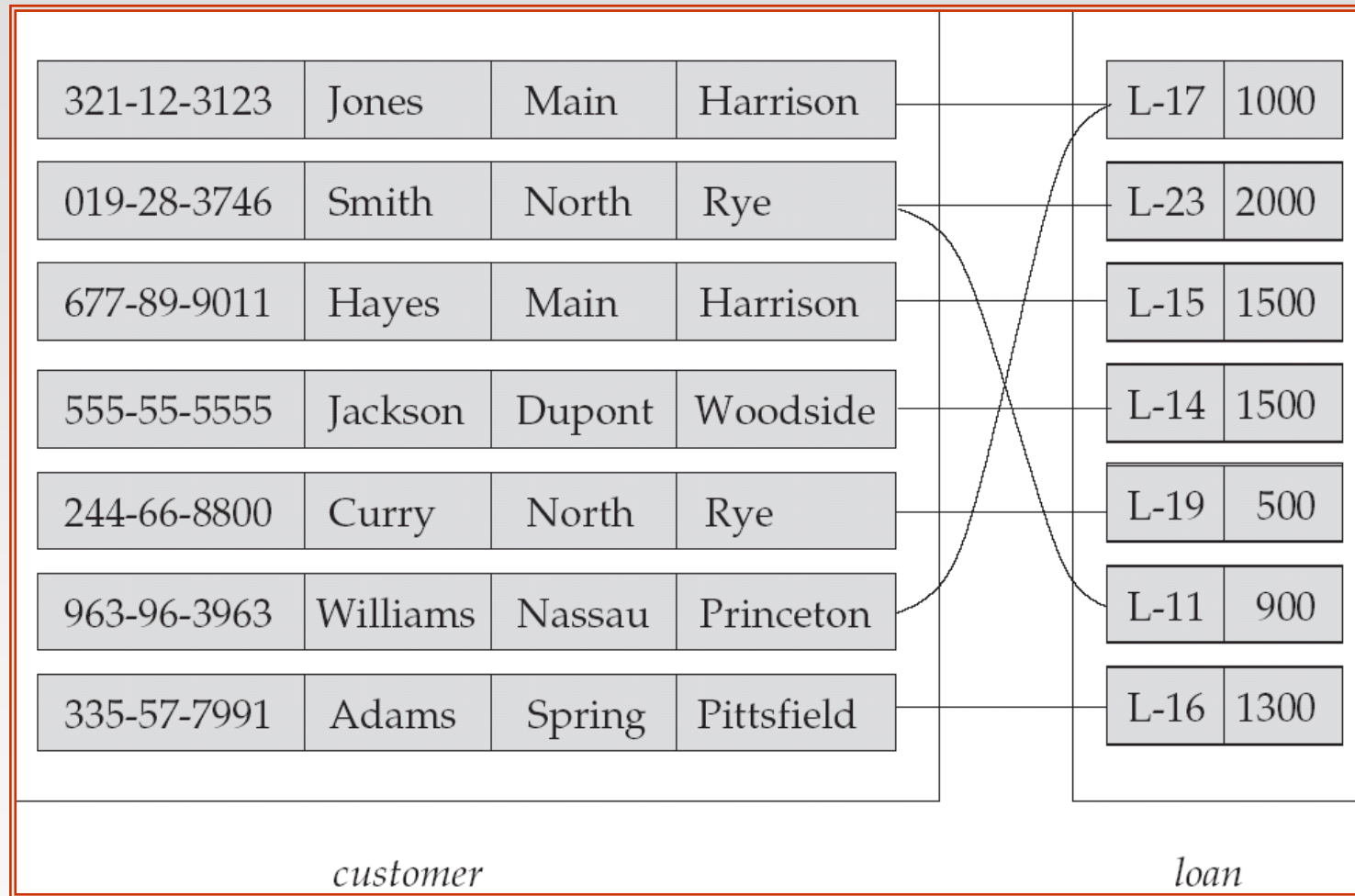
- Example:

$(\text{Hayes}, \text{A-102}) \in \text{depositor}$





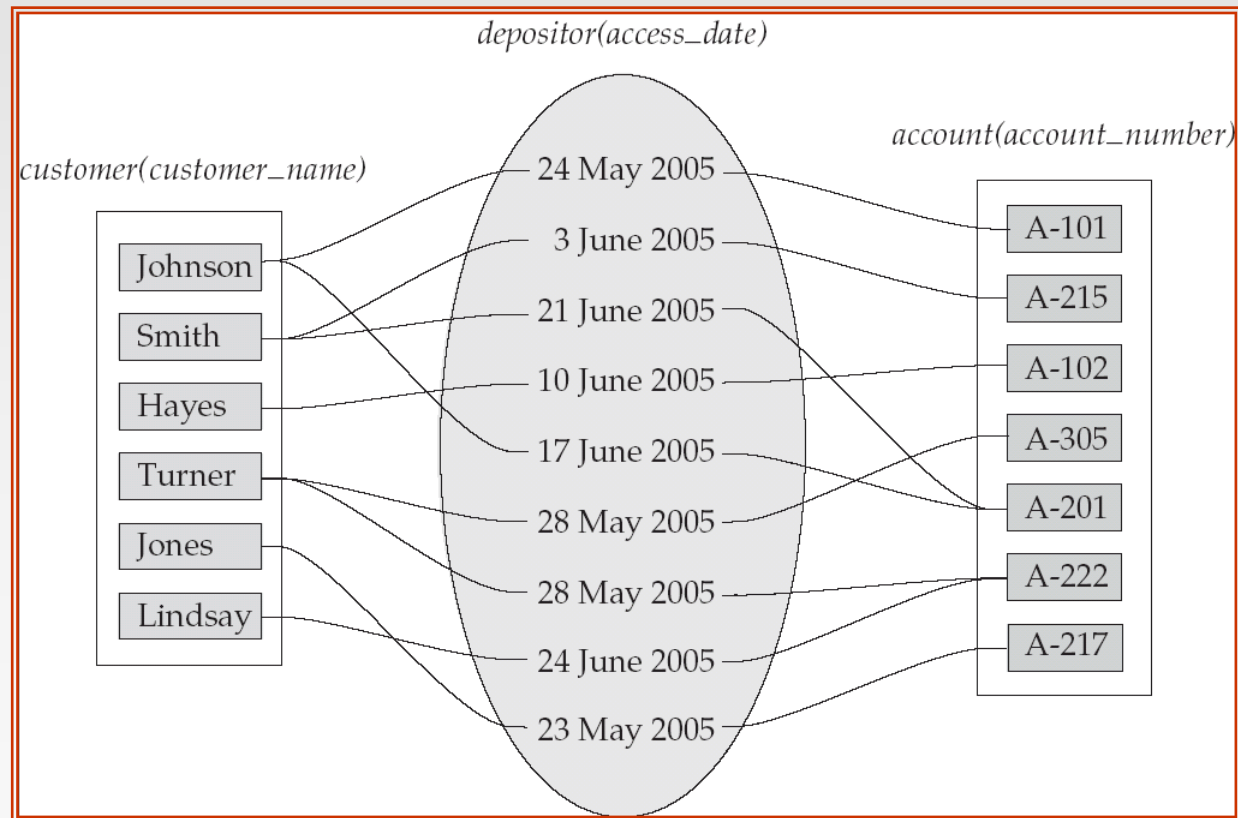
Relationship Set *borrower*





Relationship Sets (Cont.)

- An **attribute** can also be property of a relationship set.
- For instance, the *depositor* relationship set between entity sets *customer* and *account* may have the attribute *access-date*





Degree of a Relationship Set

- Refers to number of entity sets that participate in a relationship set.
- Relationship sets that involve two entity sets are **binary** (or degree two). Generally, most relationship sets in a database system are binary.
- Relationship sets may involve more than two entity sets.
 - ▶ Example: Suppose employees of a bank may have jobs (responsibilities) at multiple branches, with different jobs at different branches. Then there is a ternary relationship set between entity sets *employee*, *job*, and *branch*
- Relationships between more than two entity sets are rare. Most relationships are binary. (More on this later.)





Attributes

- An entity is represented by a set of attributes, that is descriptive properties possessed by all members of an entity set.

Example:

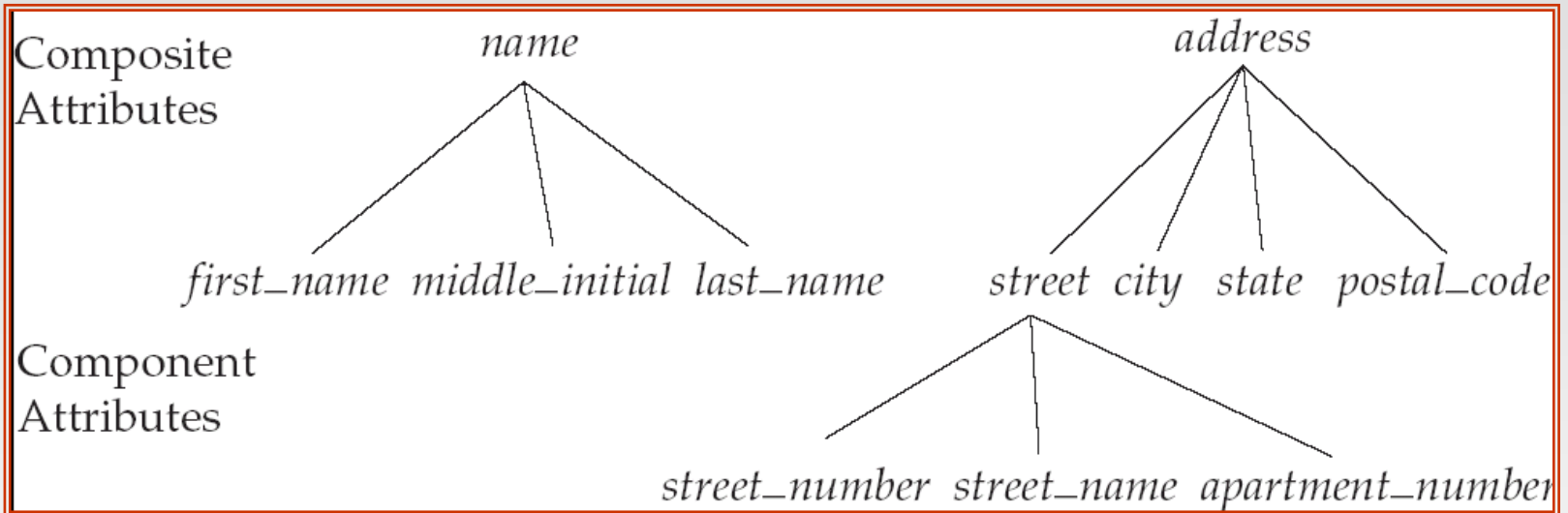
*customer = (customer_id, customer_name,
customer_street, customer_city)*
loan = (loan_number, amount)

- **Domain** – the set of permitted values for each attribute
- Attribute types:
 - *Simple and composite* attributes.
 - *Single-valued and multi-valued* attributes
 - ▶ Example: multivalued attribute: *phone_numbers*
 - *Derived* attributes
 - ▶ Can be computed from other attributes
 - ▶ Example: age, given date_of_birth





Composite Attributes





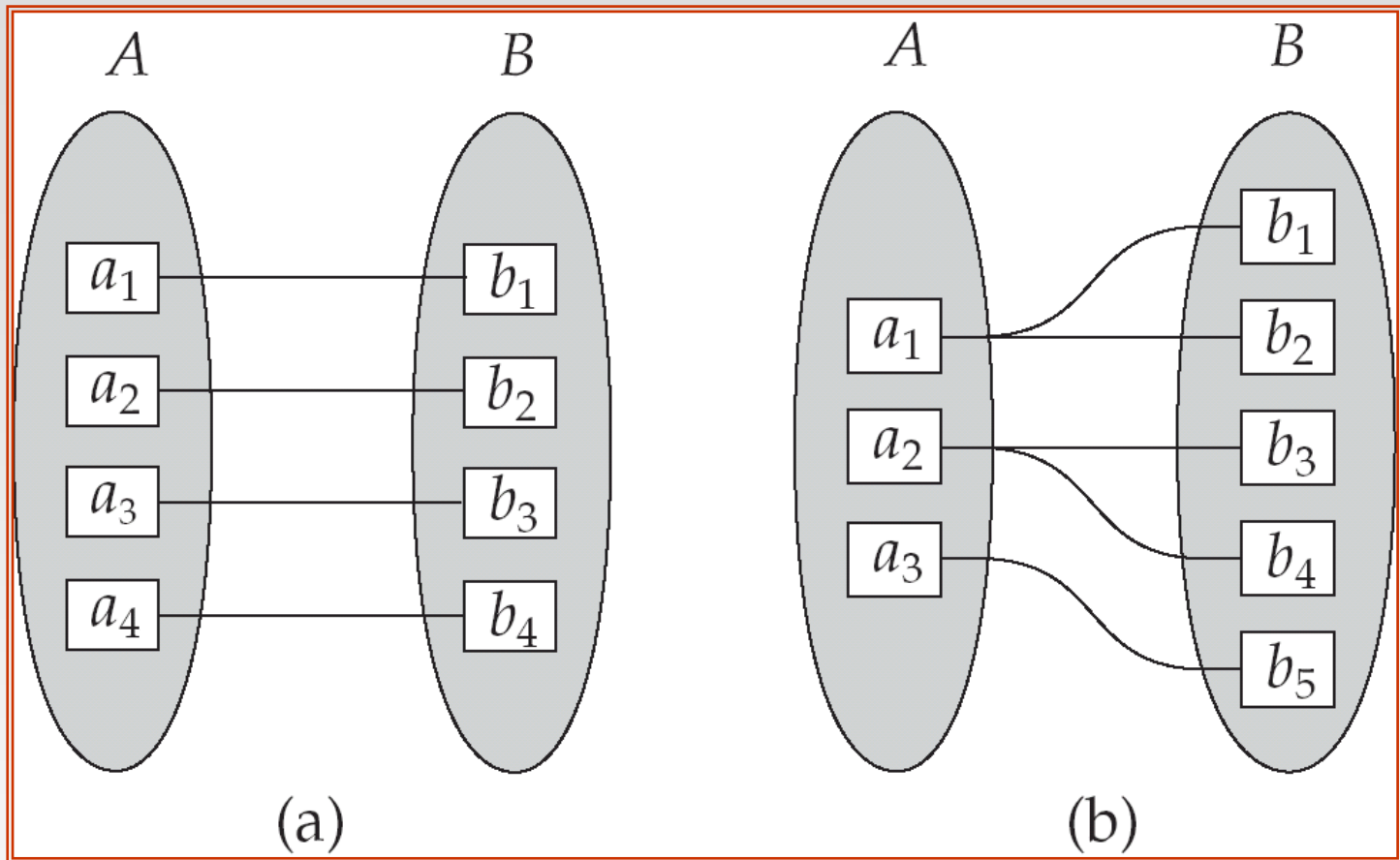
Mapping Cardinality Constraints

- Express the number of entities to which another entity can be associated via a relationship set.
- Most useful in describing binary relationship sets.
- For a binary relationship set the mapping cardinality must be one of the following types:
 - One to one
 - One to many
 - Many to one
 - Many to many





Mapping Cardinalities



One to one

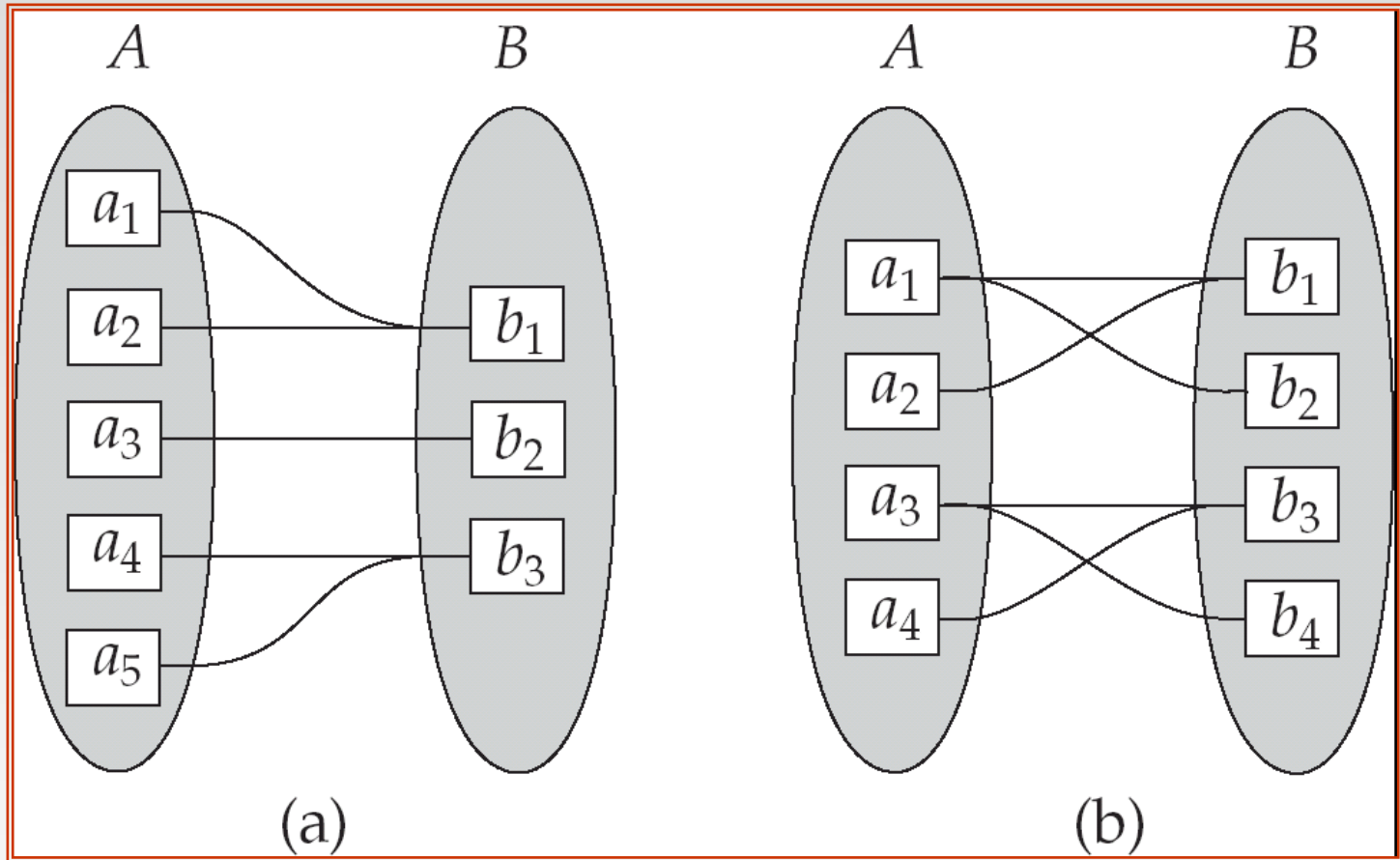
One to many

Note: Some elements in A and B may not be mapped to any elements in the other set





Mapping Cardinalities



Many to one

Many to many

Note: Some elements in A and B may not be mapped to any elements in the other set





Keys

- A **super key** of an entity set is a set of one or more attributes whose values uniquely determine each entity.
- A **candidate key** of an entity set is a minimal super key
 - *Customer_id* is candidate key of *customer*
 - *account_number* is candidate key of *account*
- Although several candidate keys may exist, one of the candidate keys is selected to be the **primary key**.





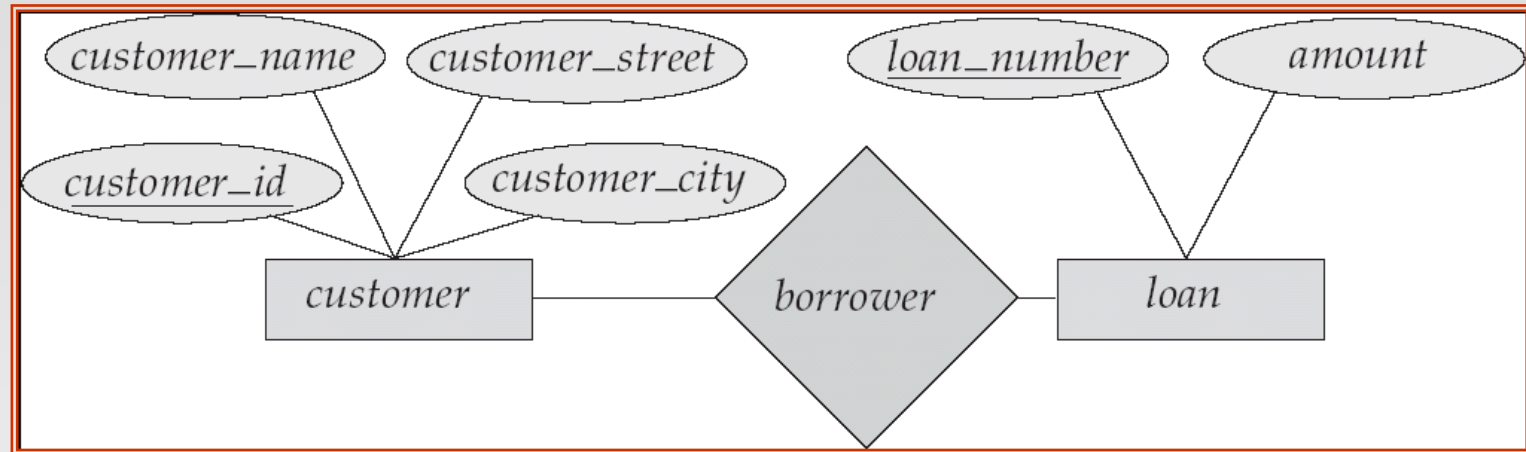
Keys for Relationship Sets

- The combination of primary keys of the participating entity sets forms a super key of a relationship set.
 - *(customer_id, account_number)* is the super key of *depositor*
 - *NOTE: this means a pair of entity sets can have at most one relationship in a particular relationship set.*
 - ▶ Example: if we wish to track all *access_dates* to each account by each customer, we cannot assume a relationship for each access. We can use a multivalued attribute though
- Must consider the mapping cardinality of the relationship set when deciding what are the candidate keys
- Need to consider semantics of relationship set in selecting the *primary key* in case of more than one candidate key





E-R Diagrams

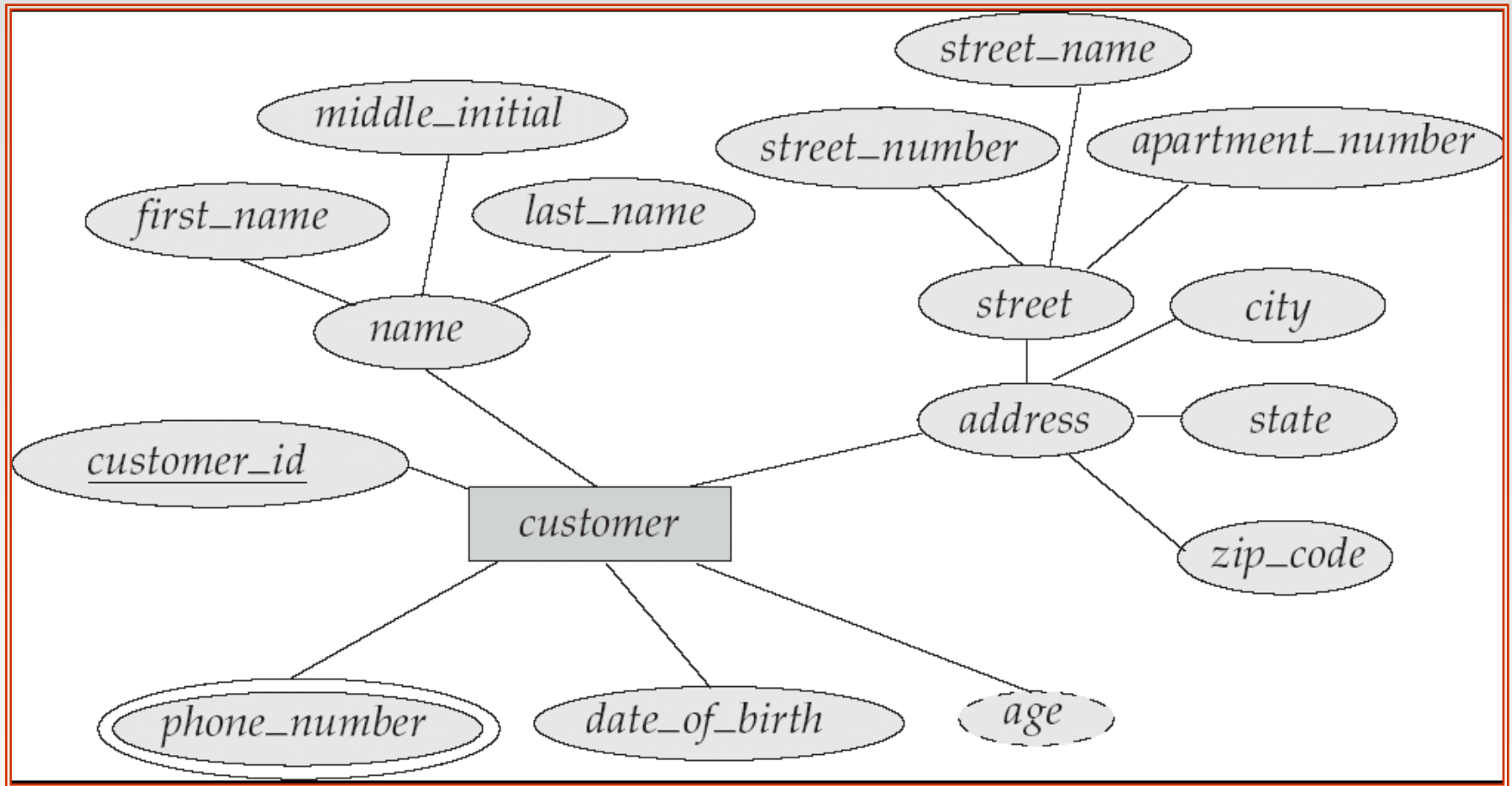


- Rectangles represent entity sets.
- Diamonds represent relationship sets.
- Lines link attributes to entity sets and entity sets to relationship sets.
- Ellipses represent attributes
 - Double ellipses represent multivalued attributes.
 - Dashed ellipses denote derived attributes.
- Underline indicates primary key attributes (will study later)



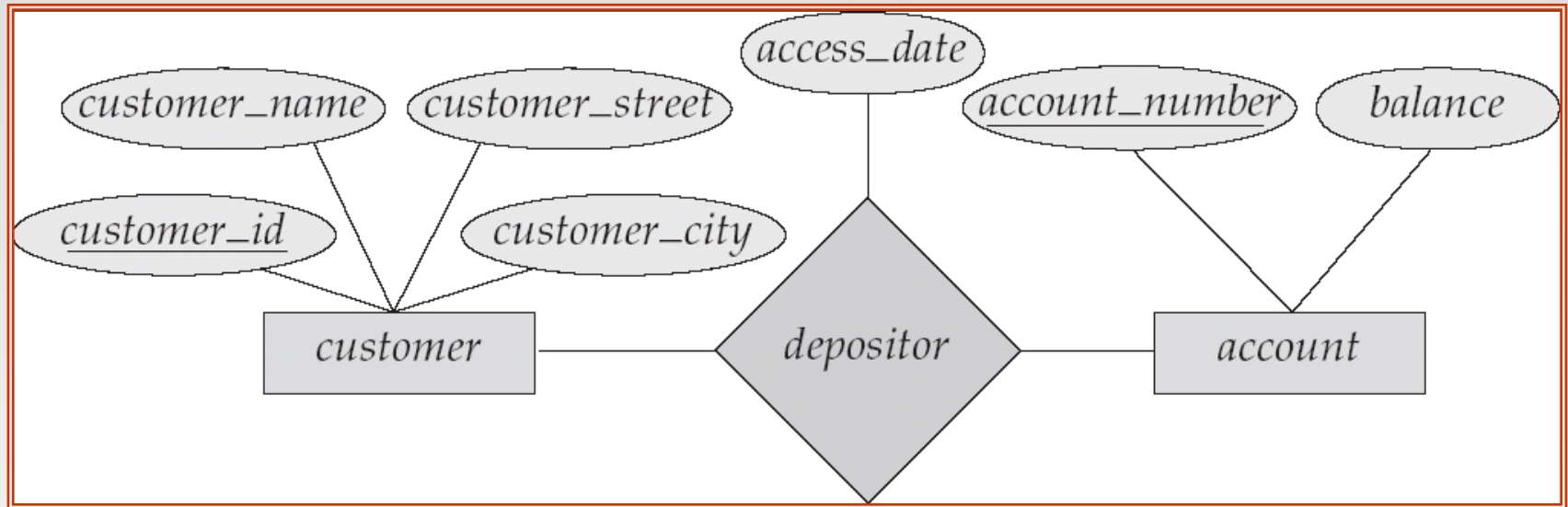


E-R Diagram With Composite, Multivalued, and Derived Attributes





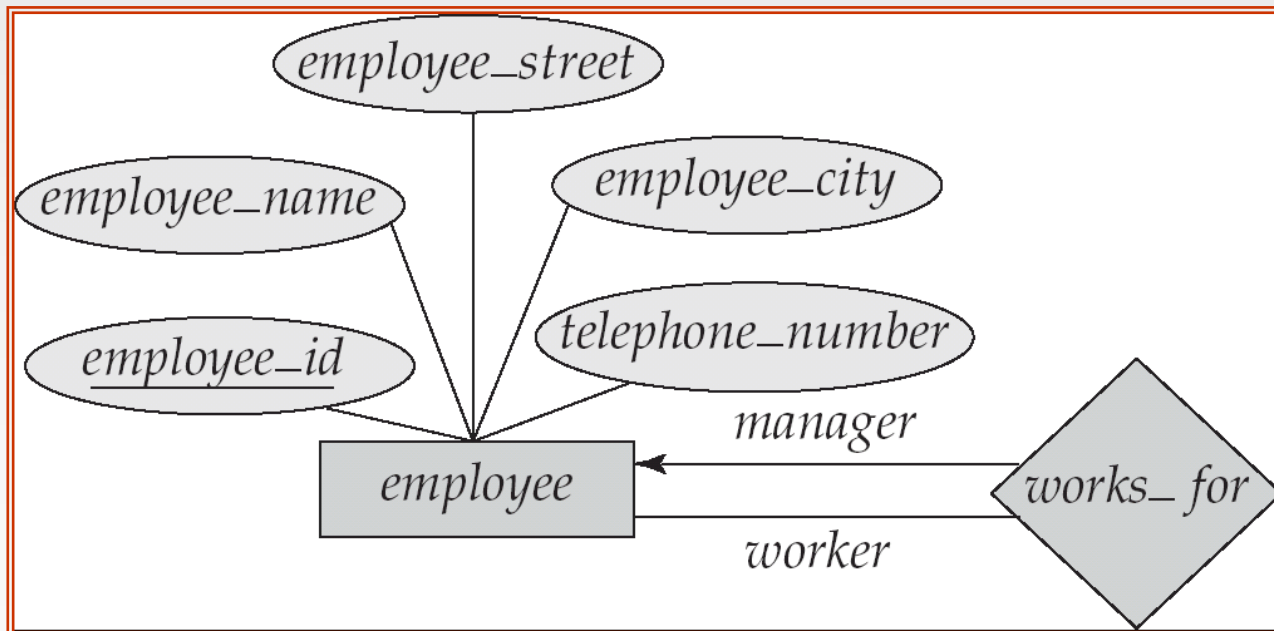
Relationship Sets with Attributes





Roles

- Entity sets of a relationship need not be distinct
- The labels “manager” and “worker” are called **roles**; they specify how employee entities interact via the works_for relationship set.
- Roles are indicated in E-R diagrams by labeling the lines that connect diamonds to rectangles.
- Role labels are optional, and are used to clarify semantics of the relationship





Cardinality Constraints

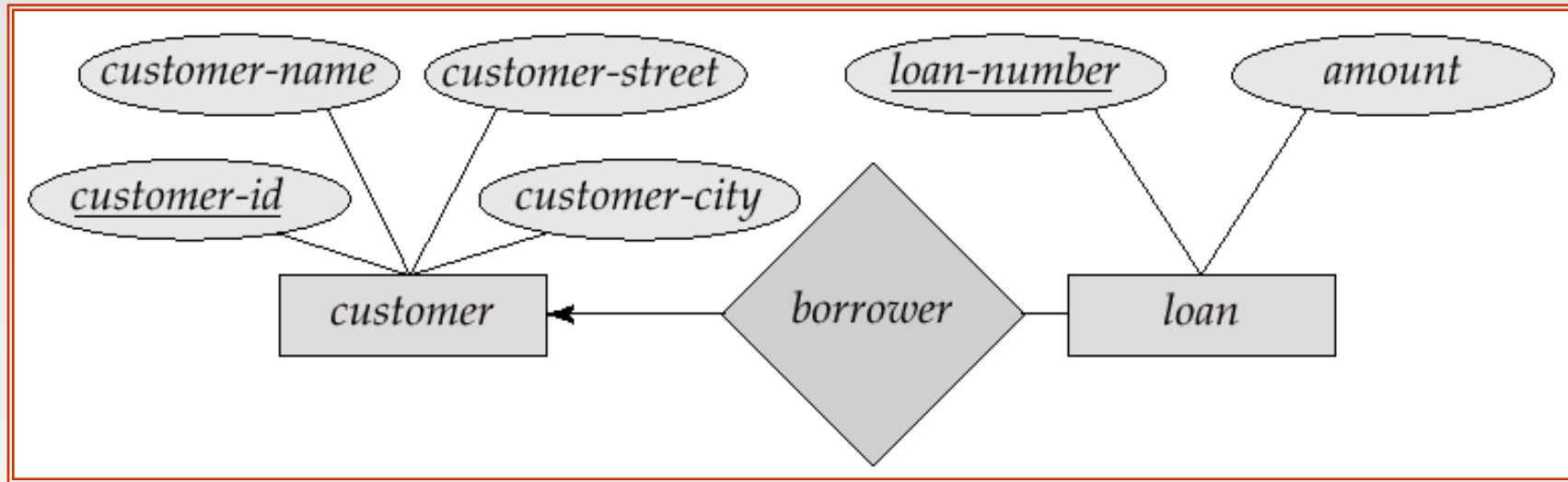
- We express cardinality constraints by drawing either a directed line ($\rightarrow$), signifying “one,” or an undirected line ($—$), signifying “many,” between the relationship set and the entity set.
- One-to-one relationship:
 - A customer is associated with at most one loan via the relationship *borrower*
 - A loan is associated with at most one customer via *borrower*





One-To-Many Relationship

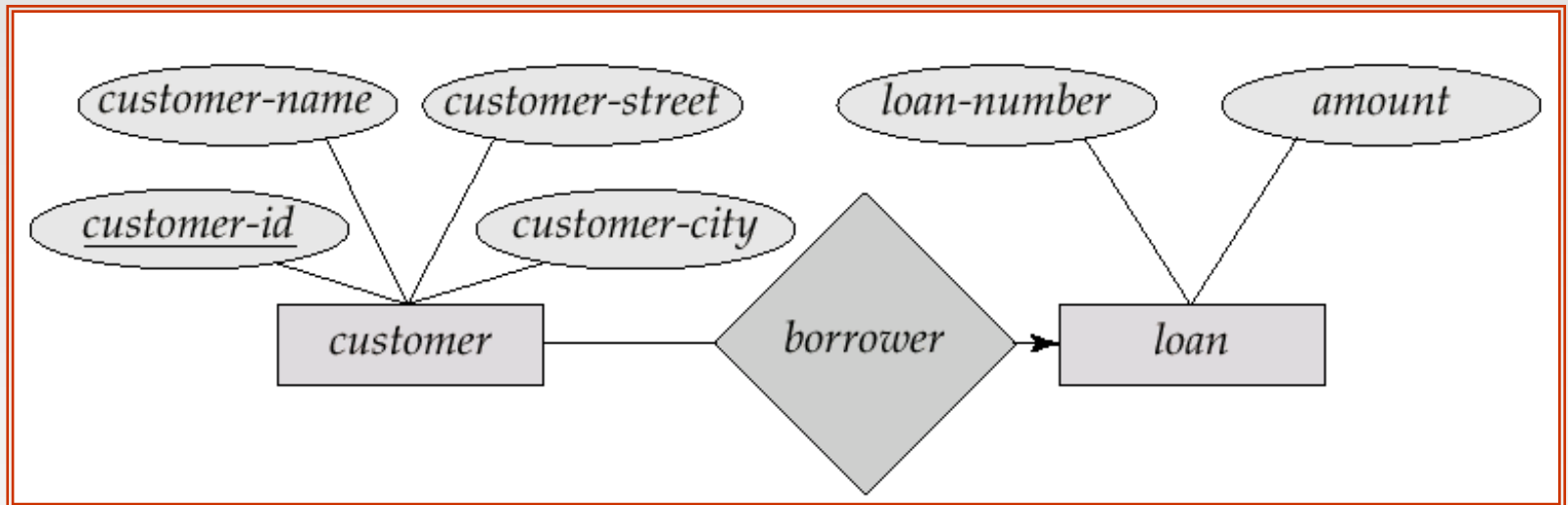
- In the one-to-many relationship a loan is associated with at most one customer via *borrower*, a customer is associated with several (including 0) loans via *borrower*





Many-To-One Relationships

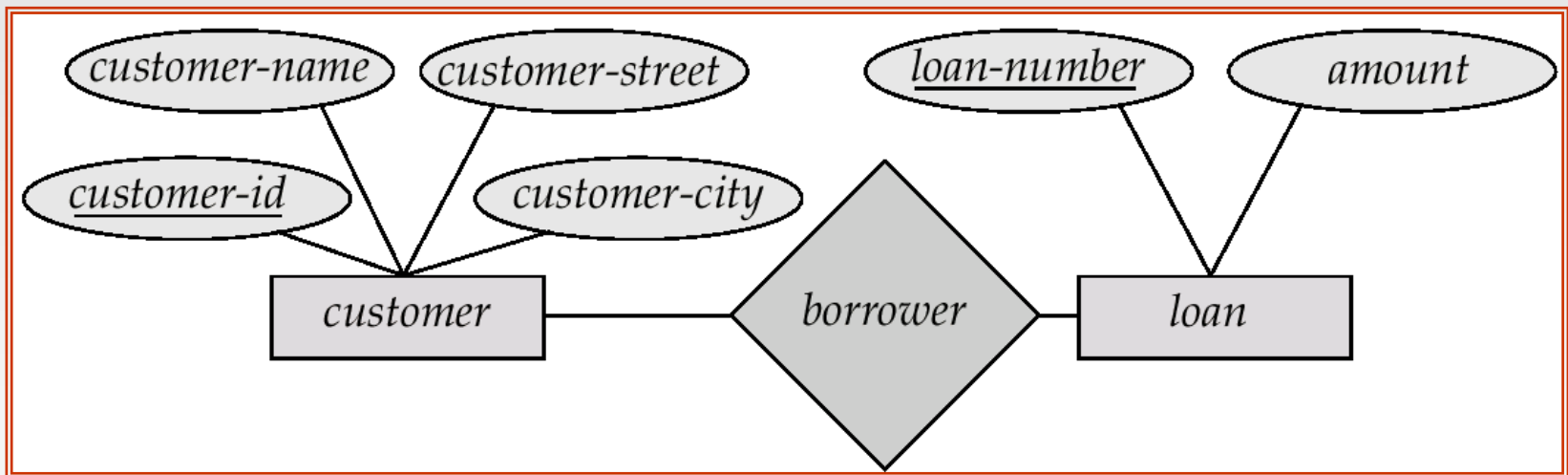
- In a many-to-one relationship a loan is associated with several (including 0) customers via *borrower*, a customer is associated with at most one loan via *borrower*





Many-To-Many Relationship

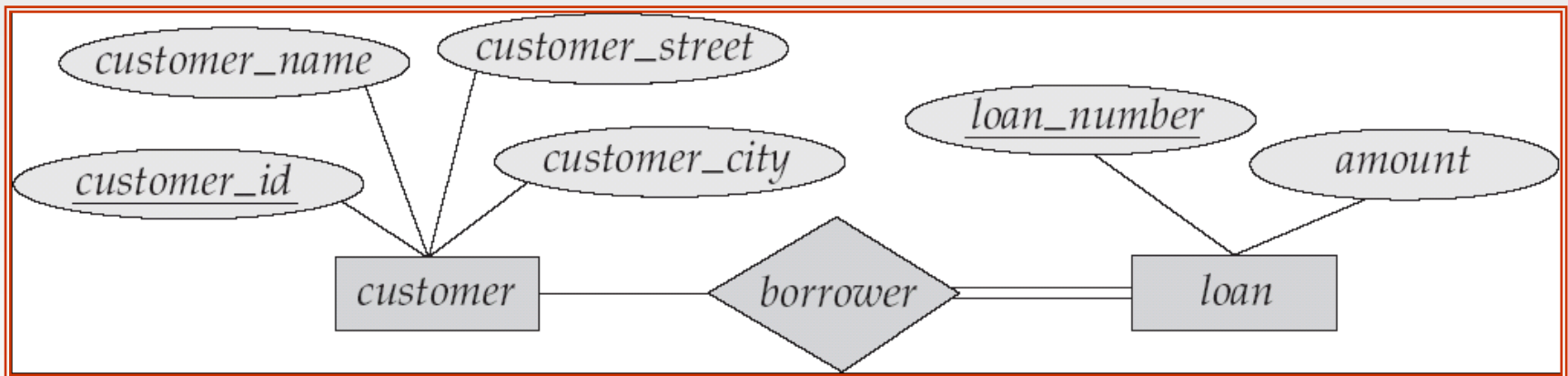
- A customer is associated with several (possibly 0) loans via borrower
- A loan is associated with several (possibly 0) customers via borrower





Participation of an Entity Set in a Relationship Set

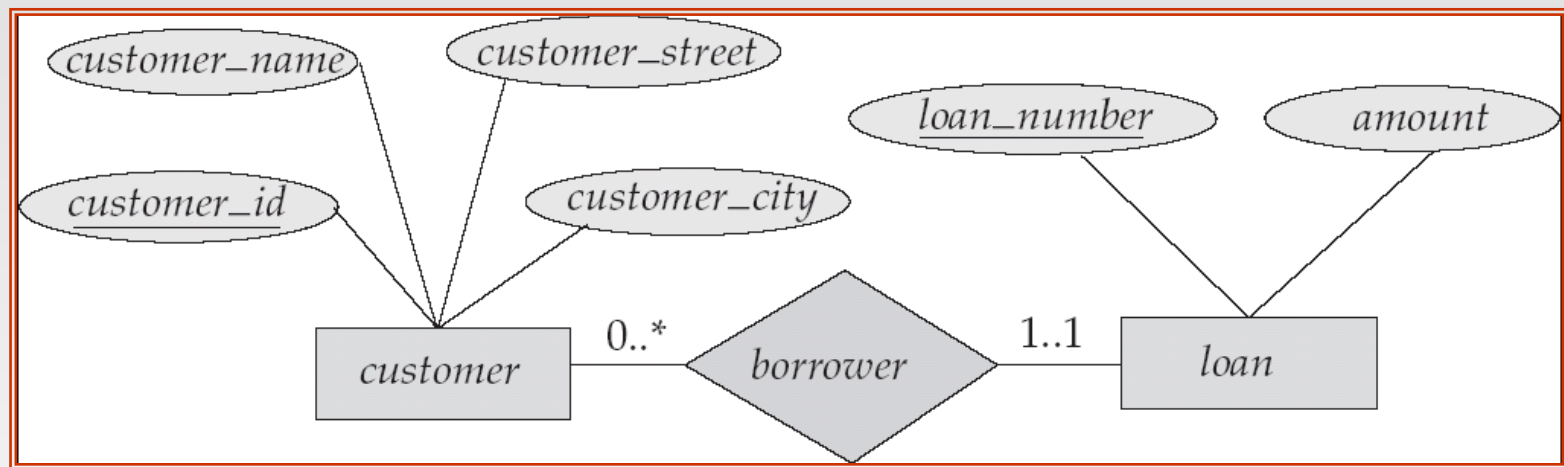
- Total participation (indicated by double line): every entity in the entity set participates in at least one relationship in the relationship set
 - E.g. participation of loan in borrower is total
 - ▶ every loan must have a customer associated to it via borrower
- Partial participation: some entities may not participate in any relationship in the relationship set
 - Example: participation of customer in borrower is partial





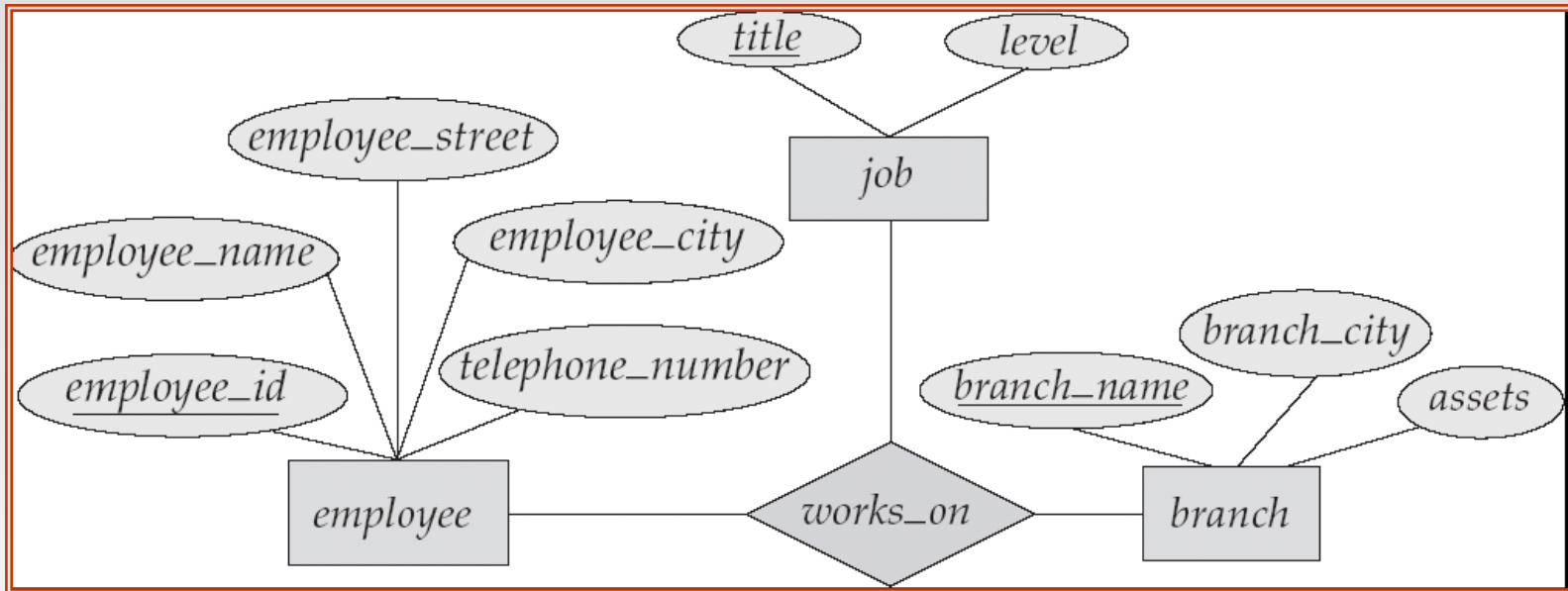
Alternative Notation for Cardinality Limits

- Cardinality limits can also express participation constraints





E-R Diagram with a Ternary Relationship





Cardinality Constraints on Ternary Relationship

- We allow at most one arrow out of a ternary (or greater degree) relationship to indicate a cardinality constraint
- E.g. an arrow from *works_on* to *job* indicates each employee works on at most one job at any branch.
- If there is more than one arrow, there are two ways of defining the meaning.
 - E.g a ternary relationship R between A , B and C with arrows to B and C could mean
 1. each A entity is associated with a unique entity from B and C or
 2. each pair of entities from (A, B) is associated with a unique C entity, and each pair (A, C) is associated with a unique B
 - Each alternative has been used in different formalisms
 - To avoid confusion we outlaw more than one arrow





Design Issues

- **Use of entity sets vs. attributes**

Choice mainly depends on the structure of the enterprise being modeled, and on the semantics associated with the attribute in question.

- **Use of entity sets vs. relationship sets**

Possible guideline is to designate a relationship set to describe an action that occurs between entities

- **Binary versus n -ary relationship sets**

Although it is possible to replace any nonbinary (n -ary, for $n > 2$) relationship set by a number of distinct binary relationship sets, a n -ary relationship set shows more clearly that several entities participate in a single relationship.

- **Placement of relationship attributes**





Binary Vs. Non-Binary Relationships

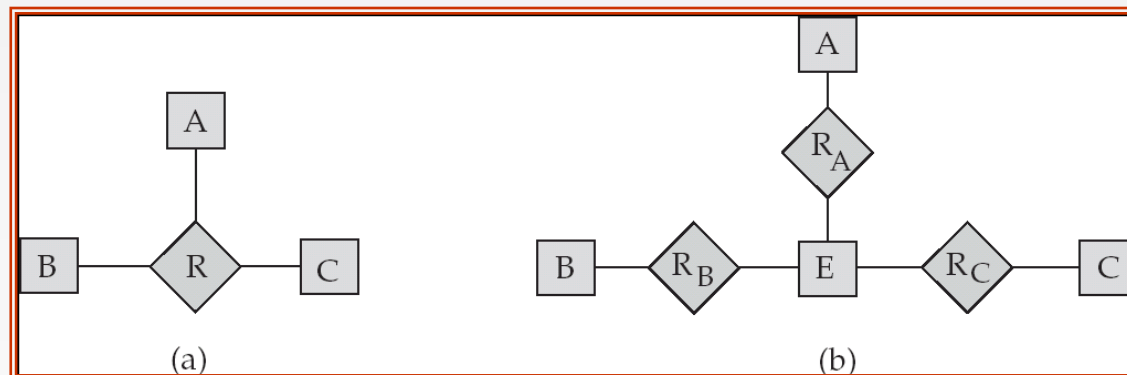
- Some relationships that appear to be non-binary may be better represented using binary relationships
 - E.g. A ternary relationship *parents*, relating a child to his/her father and mother, is best replaced by two binary relationships, *father* and *mother*
 - ▶ Using two binary relationships allows partial information (e.g. only mother being know)
 - But there are some relationships that are naturally non-binary
 - ▶ Example: *works_on*





Converting Non-Binary Relationships to Binary Form

- In general, any non-binary relationship can be represented using binary relationships by creating an artificial entity set.
 - Replace R between entity sets A , B and C by an entity set E , and three relationship sets:
 1. R_A , relating E and A
 2. R_B , relating E and B
 3. R_C , relating E and C
 - Create a special identifying attribute for E
 - Add any attributes of R to E
 - For each relationship (a_i, b_i, c_i) in R , create
 1. a new entity e_i in the entity set E
 2. add (e_i, a_i) to R_A
 3. add (e_i, b_i) to R_B
 4. add (e_i, c_i) to R_C





Converting Non-Binary Relationships (Cont.)

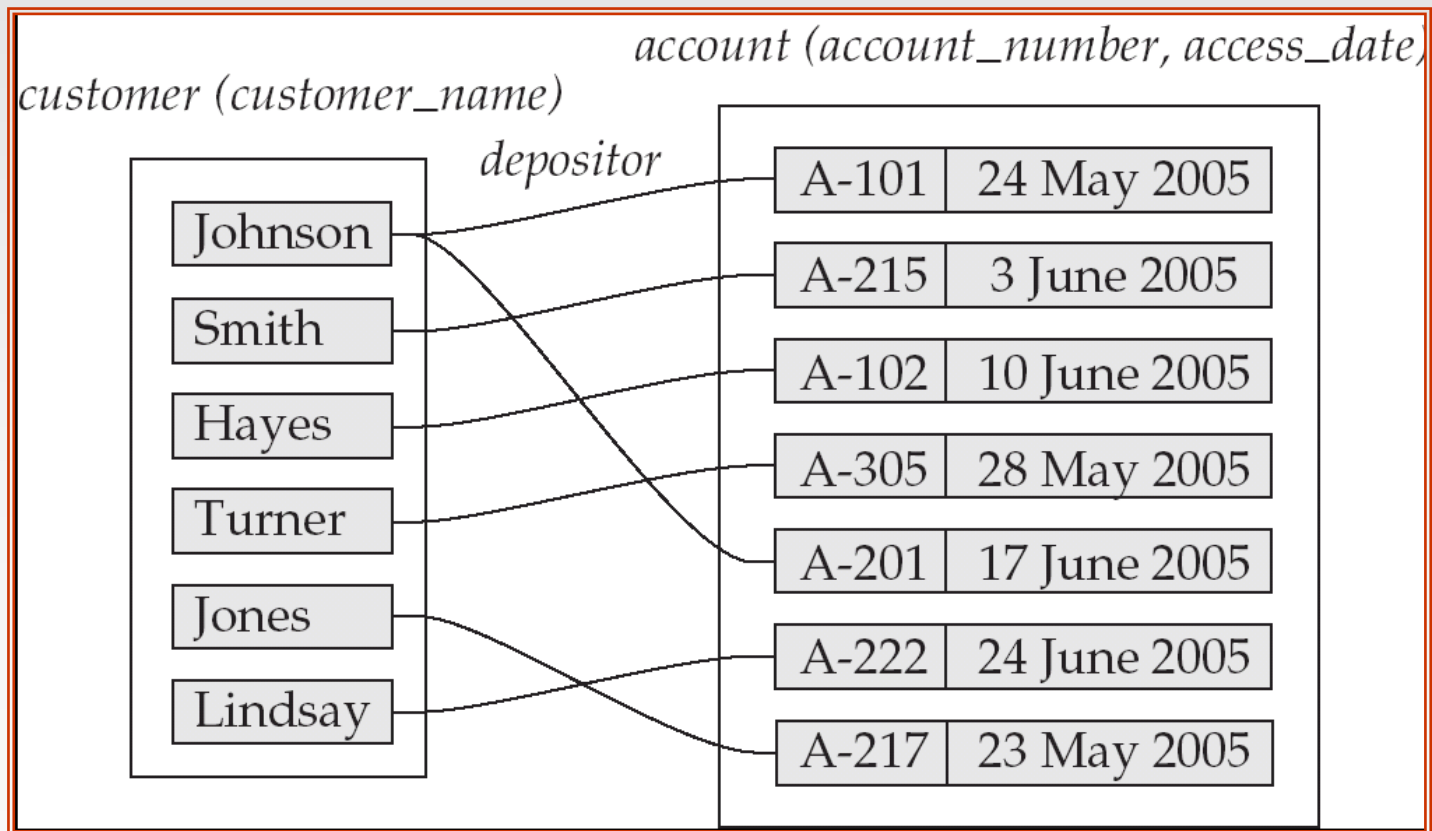
- Also need to translate constraints
 - Translating all constraints may not be possible
 - There may be instances in the translated schema that cannot correspond to any instance of R
 - ▶ Exercise: *add constraints to the relationships R_A , R_B and R_C to ensure that a newly created entity corresponds to exactly one entity in each of entity sets A , B and C*
 - We can avoid creating an identifying attribute by making E a weak entity set (described shortly) identified by the three relationship sets





Mapping Cardinalities affect ER Design

- Can make access-date an attribute of account, instead of a relationship attribute, if each account can have only one customer
 - That is, the relationship from account to customer is many to one, or equivalently, customer to account is one to many





**How about doing an ER design
interactively on the board?
Suggest an application to be modeled.**

Database System Concepts, 5th Ed.

©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use





Weak Entity Sets

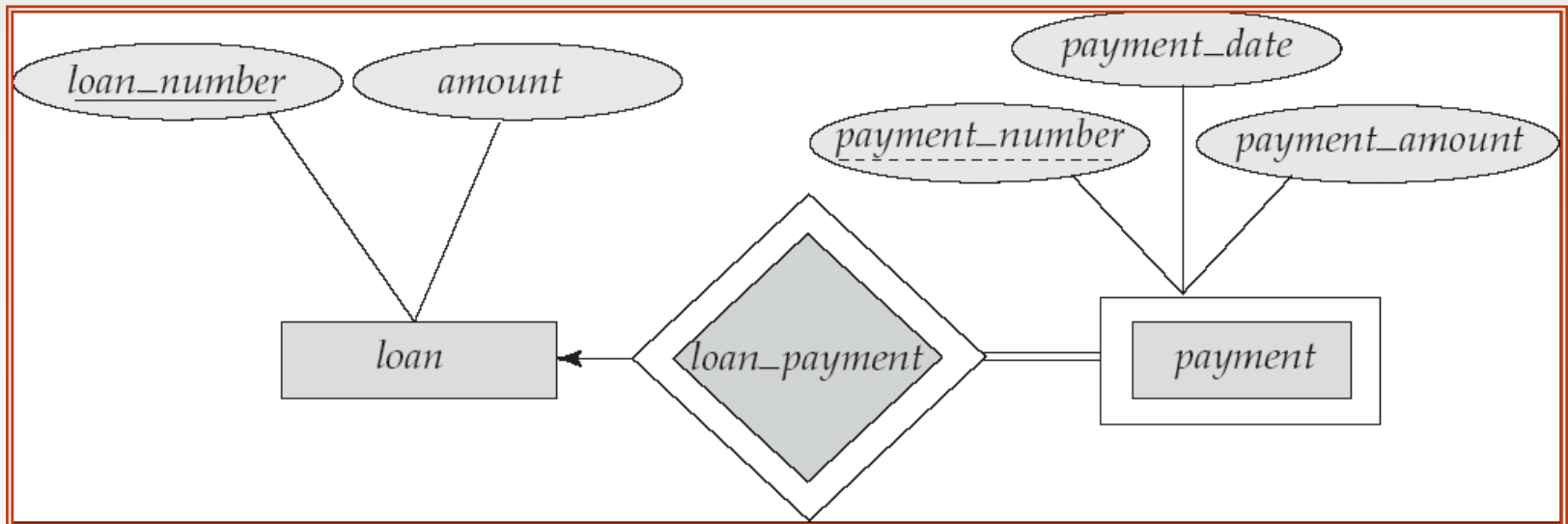
- An entity set that does not have a primary key is referred to as a **weak entity set**.
- The existence of a weak entity set depends on the existence of a **identifying entity set**
 - it must relate to the identifying entity set via a total, one-to-many relationship set from the identifying to the weak entity set
 - **Identifying relationship** depicted using a double diamond
- The **discriminator** (*or partial key*) of a weak entity set is the set of attributes that distinguishes among all the entities of a weak entity set.
- The primary key of a weak entity set is formed by the primary key of the strong entity set on which the weak entity set is existence dependent, plus the weak entity set's discriminator.





Weak Entity Sets (Cont.)

- We depict a weak entity set by double rectangles.
- We underline the discriminator of a weak entity set with a dashed line.
- `payment_number` – discriminator of the *payment* entity set
- Primary key for *payment* – (`loan_number`, `payment_number`)





Weak Entity Sets (Cont.)

- Note: the primary key of the strong entity set is not explicitly stored with the weak entity set, since it is implicit in the identifying relationship.
- If *loan_number* were explicitly stored, *payment* could be made a strong entity, but then the relationship between *payment* and *loan* would be duplicated by an implicit relationship defined by the attribute *loan_number* common to *payment* and *loan*





More Weak Entity Set Examples

- In a university, a *course* is a strong entity and a *course_offering* can be modeled as a weak entity
- The discriminator of *course_offering* would be *semester* (including year) and *section_number* (if there is more than one section)
- If we model *course_offering* as a strong entity we would model *course_number* as an attribute.

Then the relationship with *course* would be implicit in the *course_number* attribute





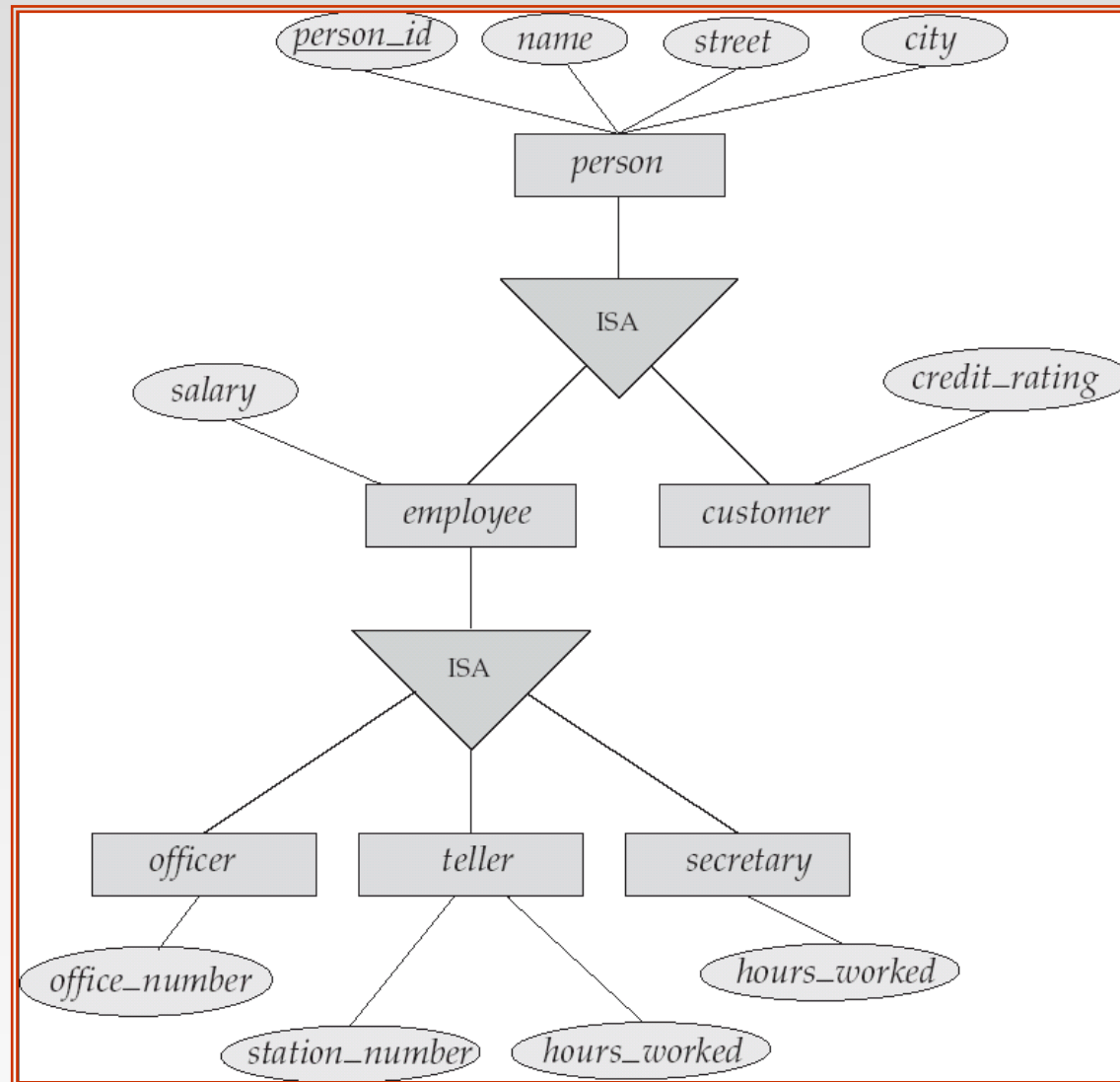
Extended E-R Features: Specialization

- Top-down design process; we designate subgroupings within an entity set that are distinctive from other entities in the set.
- These subgroupings become lower-level entity sets that have attributes or participate in relationships that do not apply to the higher-level entity set.
- Depicted by a *triangle* component labeled ISA (E.g. *customer* “is a” *person*).
- **Attribute inheritance** – a lower-level entity set inherits all the attributes and relationship participation of the higher-level entity set to which it is linked.





Specialization Example





Extended ER Features: Generalization

- **A bottom-up design process** – combine a number of entity sets that share the same features into a higher-level entity set.
- Specialization and generalization are simple inversions of each other; they are represented in an E-R diagram in the same way.
- The terms specialization and generalization are used interchangeably.





Specialization and Generalization (Cont.)

- Can have multiple specializations of an entity set based on different features.
- E.g. *permanent_employee* vs. *temporary_employee*, in addition to *officer* vs. *secretary* vs. *teller*
- Each particular employee would be
 - a member of one of *permanent_employee* or *temporary_employee*,
 - and also a member of one of *officer*, *secretary*, or *teller*
- The ISA relationship also referred to as **superclass - subclass** relationship





Design Constraints on a Specialization/Generalization

- Constraint on which entities can be members of a given lower-level entity set.
 - condition-defined
 - ▶ Example: all customers over 65 years are members of *senior-citizen* entity set; *senior-citizen* ISA *person*.
 - user-defined
- Constraint on whether or not entities may belong to more than one lower-level entity set within a single generalization.
 - **Disjoint**
 - ▶ an entity can belong to only one lower-level entity set
 - ▶ Noted in E-R diagram by writing *disjoint* next to the ISA triangle
 - **Overlapping**
 - ▶ an entity can belong to more than one lower-level entity set





Design Constraints on a Specialization/Generalization (Cont.)

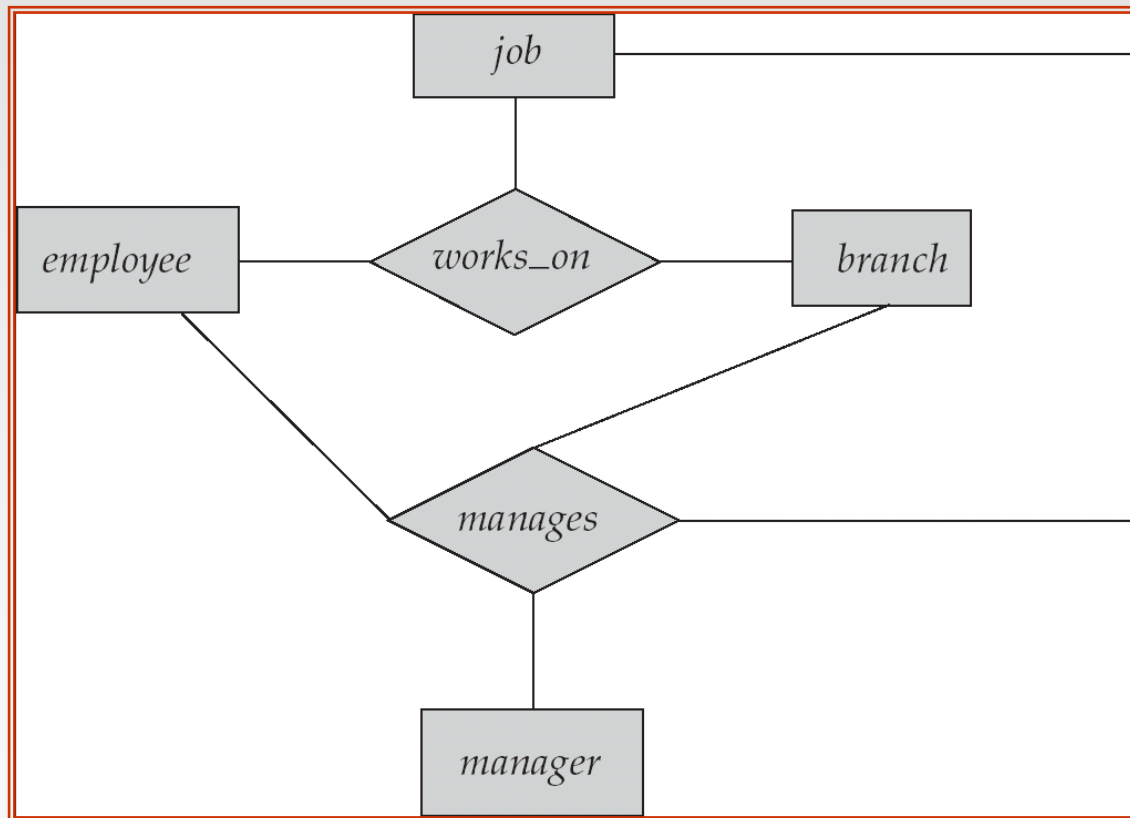
- **Completeness constraint** -- specifies whether or not an entity in the higher-level entity set must belong to at least one of the lower-level entity sets within a generalization.
 - **total** : an entity must belong to one of the lower-level entity sets
 - **partial**: an entity need not belong to one of the lower-level entity sets





Aggregation

- Consider the ternary relationship *works_on*, which we saw earlier
- Suppose we want to record managers for tasks performed by an employee at a branch





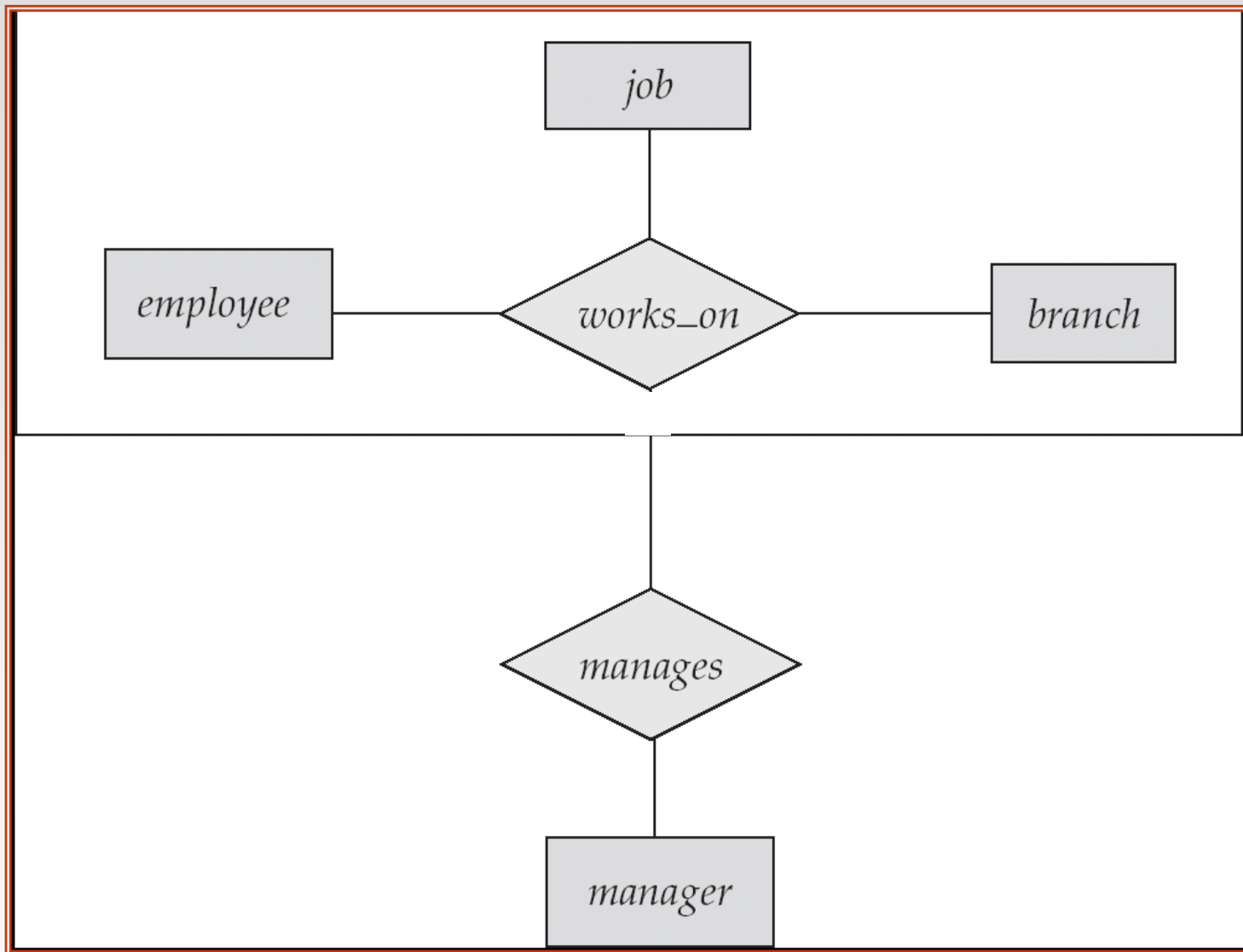
Aggregation (Cont.)

- Relationship sets *works_on* and *manages* represent overlapping information
 - Every *manages* relationship corresponds to a *works_on* relationship
 - However, some *works_on* relationships may not correspond to any *manages* relationships
 - ▶ So we can't discard the *works_on* relationship
- Eliminate this redundancy via *aggregation*
 - Treat relationship as an abstract entity
 - Allows relationships between relationships
 - Abstraction of relationship into new entity
- Without introducing redundancy, the following diagram represents:
 - An employee works on a particular job at a particular branch
 - An employee, branch, job combination may have an associated manager





E-R Diagram With Aggregation





E-R Design Decisions

- The use of an attribute or entity set to represent an object.
- Whether a real-world concept is best expressed by an entity set or a relationship set.
- The use of a ternary relationship versus a pair of binary relationships.
- The use of a strong or weak entity set.
- The use of specialization/generalization – contributes to modularity in the design.
- The use of aggregation – can treat the aggregate entity set as a single unit without concern for the details of its internal structure.





Case Study

A banking scenario

- Banks have customers.
- Customers are identified by name, custid, phone number and address.
- Customers can have one or more accounts
- Accounts are identified by an account number, account type (savings, current) and a balance.
- Customers can avail loans.
- Loans are identified by loan id, loan type (car, home, personal) and an amount.
- Banks are identified by a name, code and the address of the main office.
- Banks have branches.
- Branches are identified by a branch number, branch name and an address.
- Accounts and loans are related to the banks' branches.
- Create an ER diagram for a database to represent this application





Case Study

Solution Step 1: Identify the entities

- Bank
- Branch
- Customer
- Account
- Loan

Solution Step 2: Identify attributes of entities

Bank	Branch	Customer
Name	Branch#	Name
Code	Branch name	Custid
Address	Address	Phone
		Address
Account	Loan	
Account number	Loan id	
Account type	Loan type	
Balance	Amount	





Case Study

Solution Step 3: Identify relationships between entities

- Bank **has** Branch
- Branch **maintains** accounts
- Branch **offers** loans
- Account is **held by** customer
- Loan is **availed by** customer

Solution Step 4: Analyze cardinality of relationships

- Bank **has** Branch : A bank has many branches-> **1:N**
- Branch **maintains** accounts: One branch maintains many accounts-> **1:N**
- Branch **offers** loans : One branch offers many loans -> **1:N**
- Account is **held by** customer -> **M:N**
- Loan is **availed by** customer -> **M:N**





Case Study

Solution Step 5: Identify weak entities if any

Branch: Depends on strong entity Bank

Solution Step 6: Identify participation types

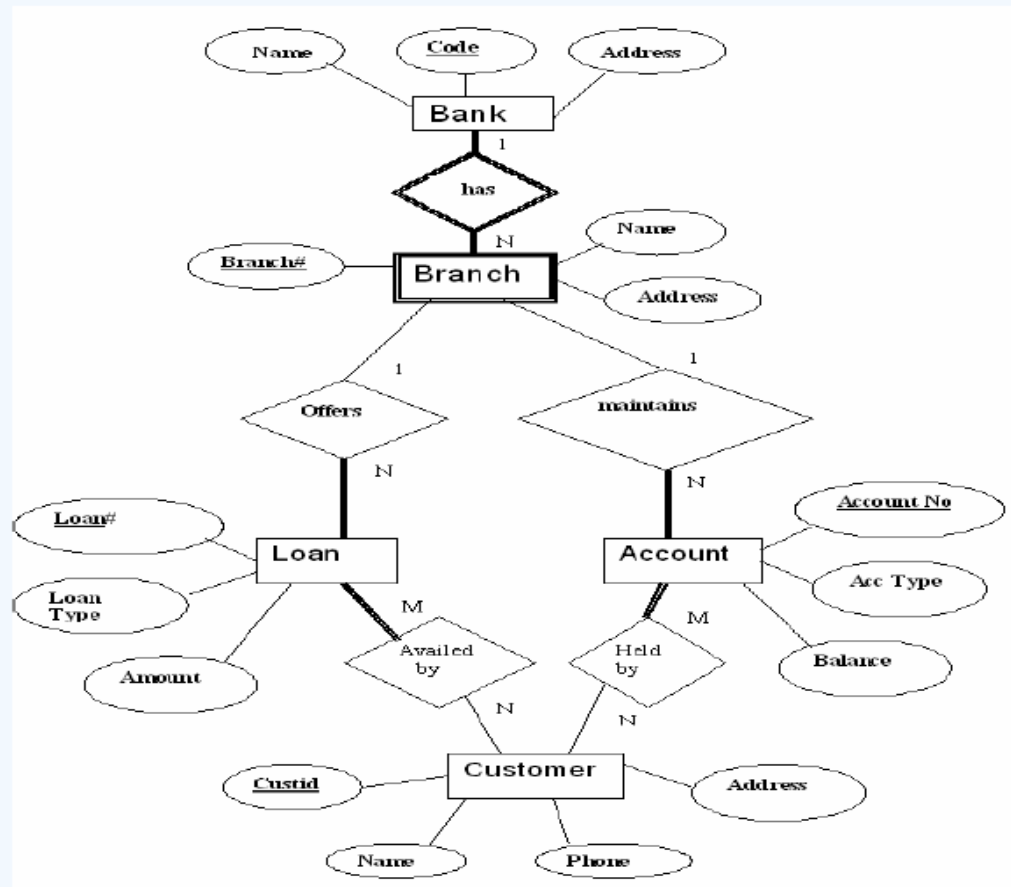
- Bank has Branch -> both **total**
- Branch maintains accounts-> Branch :partial Account: Total
- Branch offers loans -> Branch: partial Loan: Total
- Account is held by customer-> both Total
- Loan is availed by customer-> Loan : Total Customer: Partial





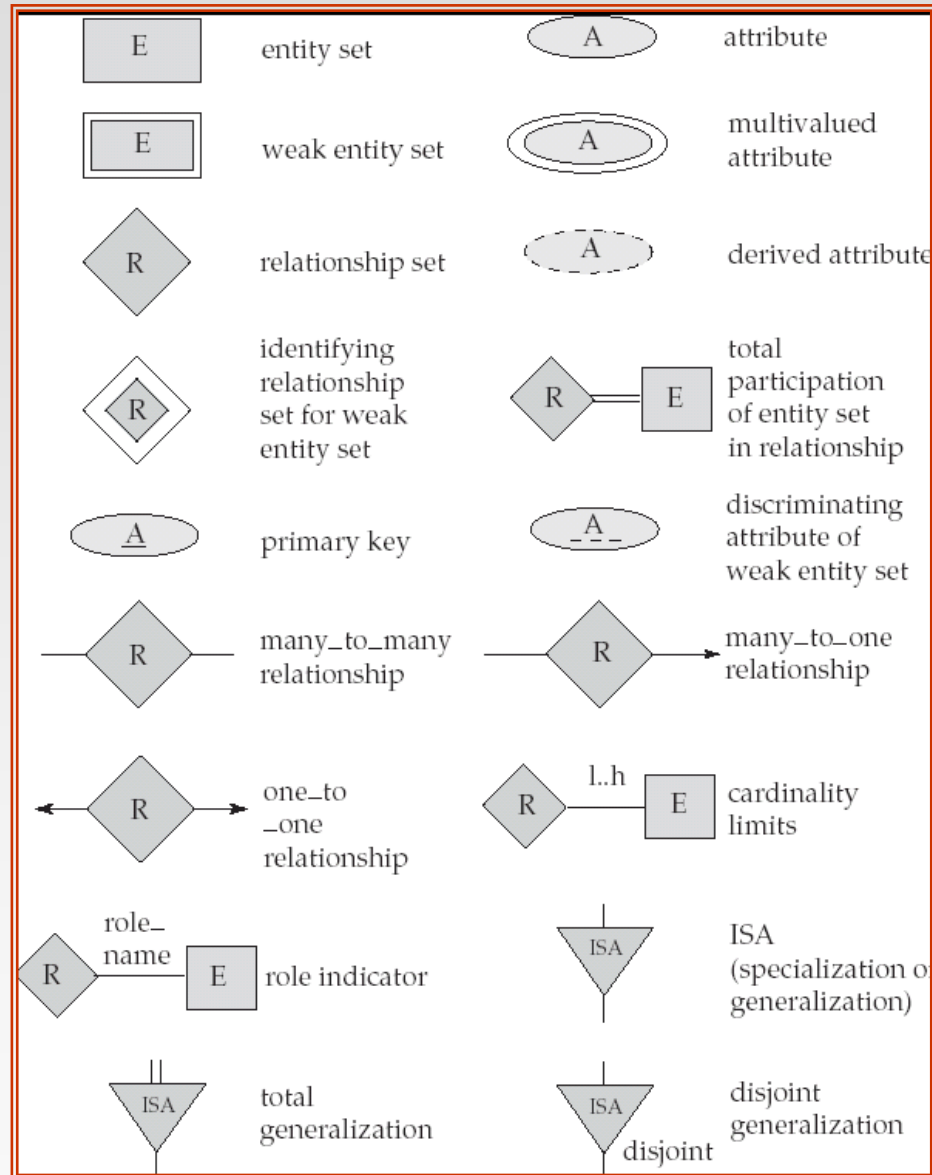
Case Study

Represented diagrammatically



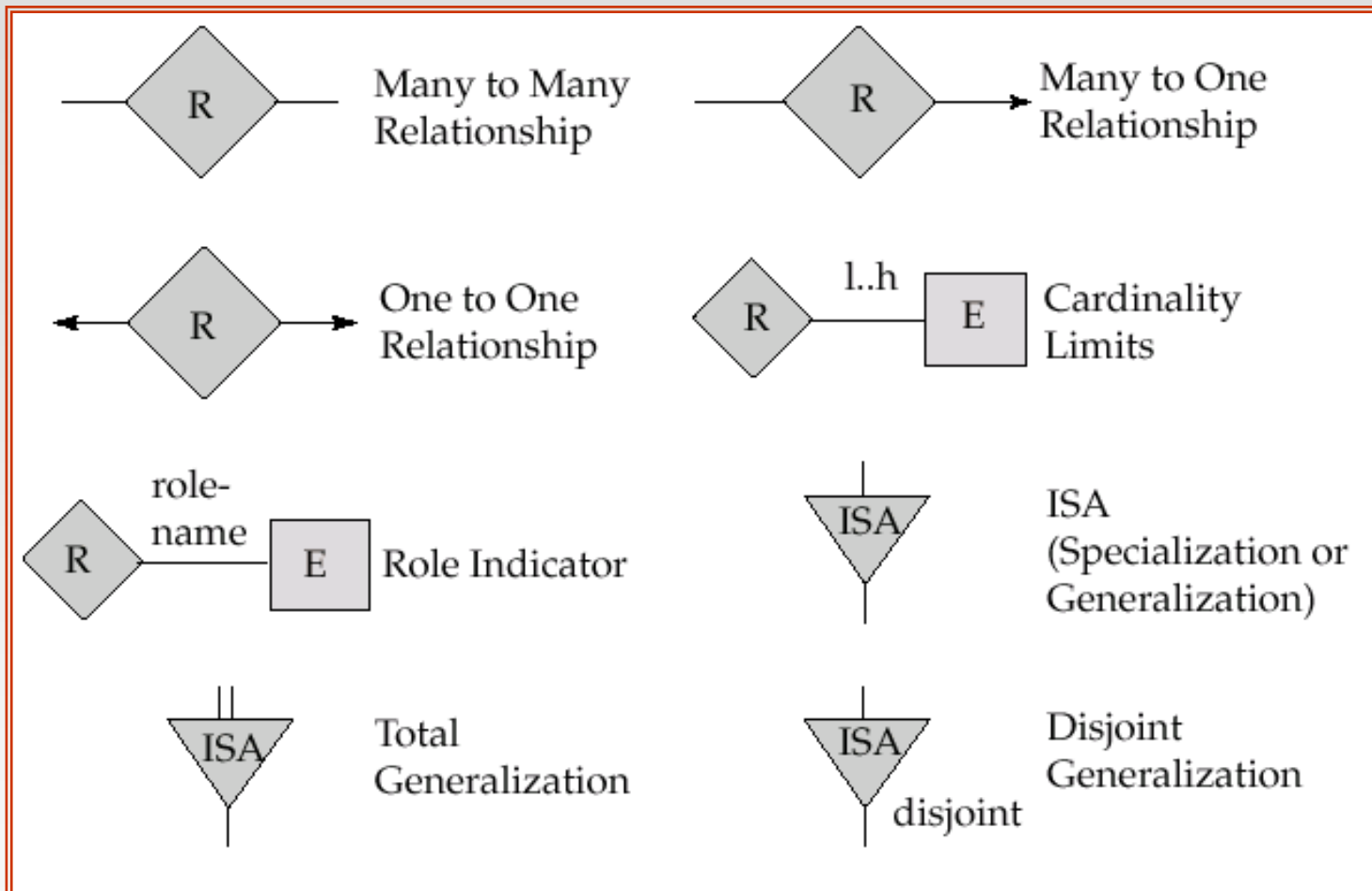


Summary of Symbols Used in E-R Notation





Summary of Symbols (Cont.)



TRANSFORM ER TO RELATIONAL SCHEMA

- This is the logical design step of the database design process.
- This step transforms ER Diagrams to relations.

From ER Diagram to Relations (1)

ER Concept	Relational Concept
Strong entity	Tuple
Weak entity	???
Strong entity set	Relation
Weak entity set	???
Attribute	Attribute
Key	Key
Composite attribute	???
Multi-valued attribute	???

From ER Diagram to Relations (2)

ER Concept	Relational Concept
Unary 1-1 relationship	???
Unary 1-m relationship	???
Unary m-m relationship	???
Binary 1-1 relationship	???
Binary 1-m relationship	???
Binary m-m relationship	???
Ternary relationship	???
IS_A hierarchy	???

From ER Diagram to Relations (3)

Summary:

- ER model is more semantically oriented than the relational model.
- Some ER concepts, e.g., connectivity constraints, cannot be expressed in the relational model.
- Are there important concepts that are supported by the relational model but not by the ER model?
 - Foreign key

Basic Ideas of the Transformation

Entity ====> Tuple

Entity set ====> Relation

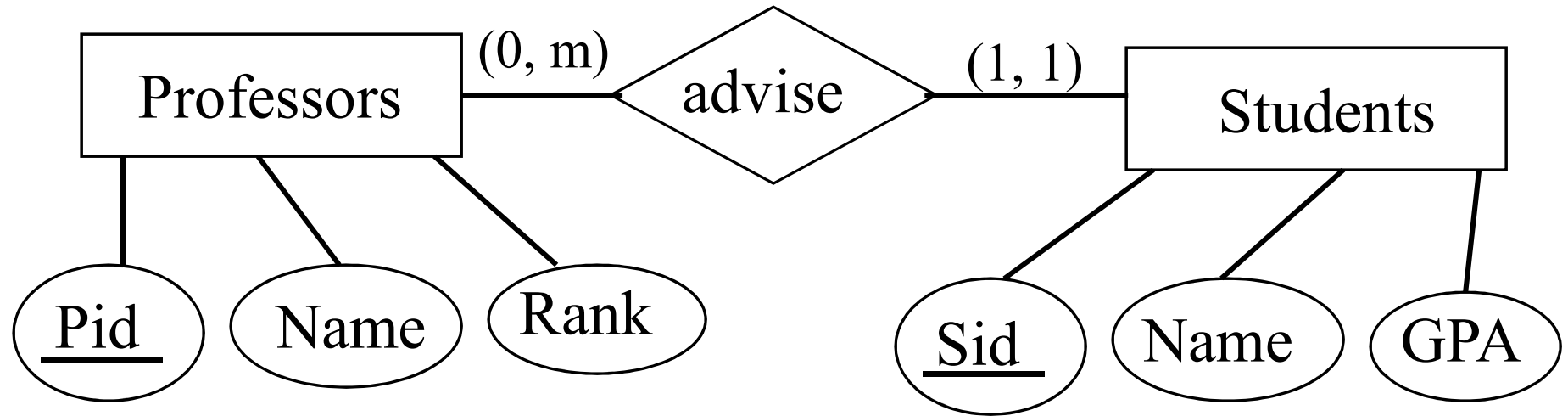
Attribute ====> Attribute

Key ====> Key

Relationship ====> Tuple or foreign key value(s)

Relationship set ====> Relation or foreign key(s)

An Example (1)



Professors	Advise	Students
p1: 123, Jack, Prof.	p1 advises s1	s1: 456, John, 3.4
p2: 234, Ann, Prof.	p1 advises s2	s2: 567, Carl, 3.2
p3: 345, Bob, Prof.	p3 advises s3	s3: 678, Ken, 3.5

An Example (2)

Transform the ER diagram into three relations:

Professors			Advise		Students		
Pid	Name	Rank	Pid	Sid	Sid	Name	GPA
123	Jack	Prof.	123	456	456	John	3.4
234	Ann	Prof.	123	567	567	Carl	3.2
345	Bob	Prof.	345	678	678	Ken	3.5

An Example (3)

Two relations are sufficient:

Professors

Pid	Name	Rank
123	Jack	Prof.
234	Ann	Prof.
345	Bob	Prof.

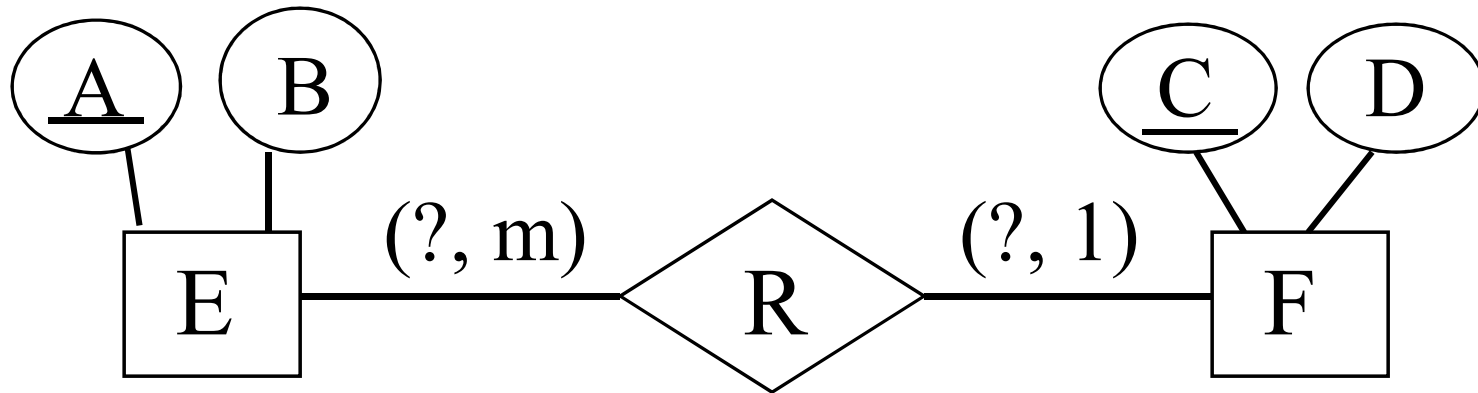
Students

Sid	Name	GPA	Pid
456	John	3.4	123
567	Carl	3.2	123
678	Ken	3.5	345

From ER Diagram to Relations

Transformation Guideline 1: Transform relationship sets to foreign key attributes whenever possible.

Transform Binary Relationship (1)

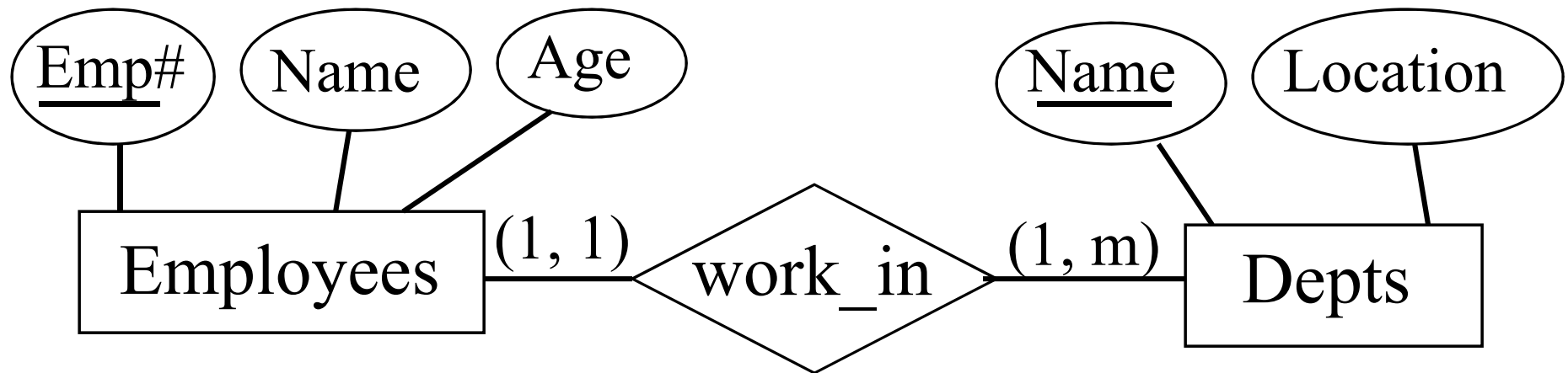


Case 1: one-to-many relationship

$\implies E(\text{A}, B), F(\text{C}, D, \text{A})$

- Relationship R is transformed to a foreign key.
- The foreign key goes to the side with $\text{max_card} = 1$

Transform Binary Relationship (2)

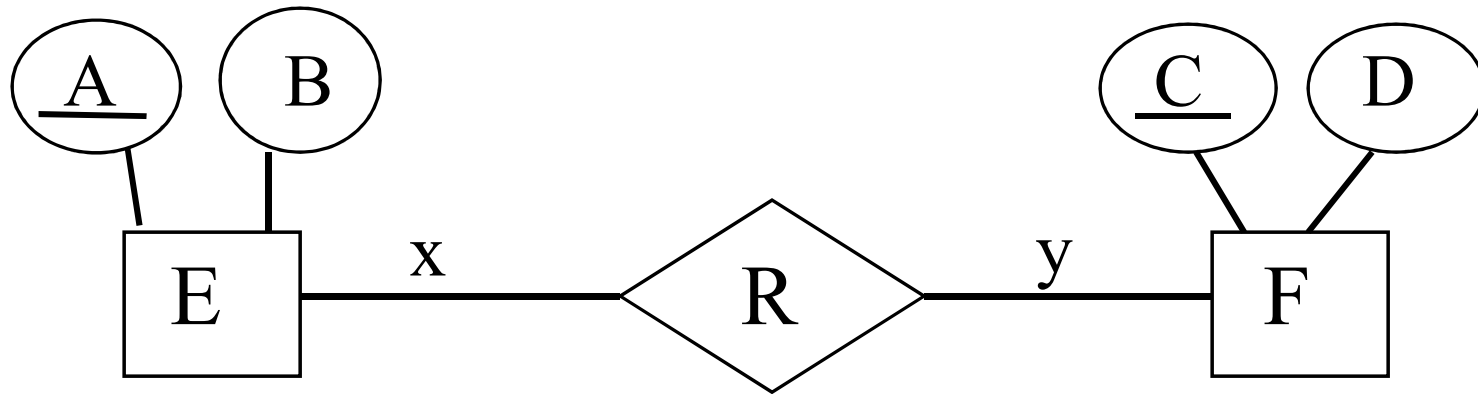


==> Depts(**Name**, Location)

Employees(**Emp#**, Name, Age, **Dept_name**)

Renaming is useful for improving understandability.

Transform Binary Relationship (3)



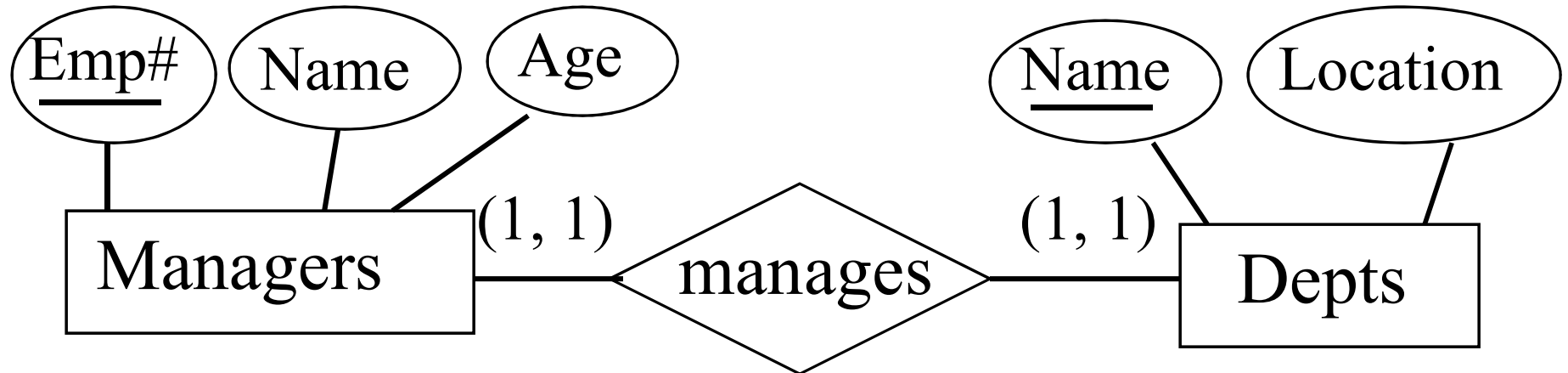
Case 2: one-to-one relationship

Case 2.1: $x = (1, 1)$ and $y = (1, 1)$

$\implies E(A, B), F(C, D, A)$ or

$\implies E(A, B, C), F(C, D)$

Transform Binary Relationship (4)



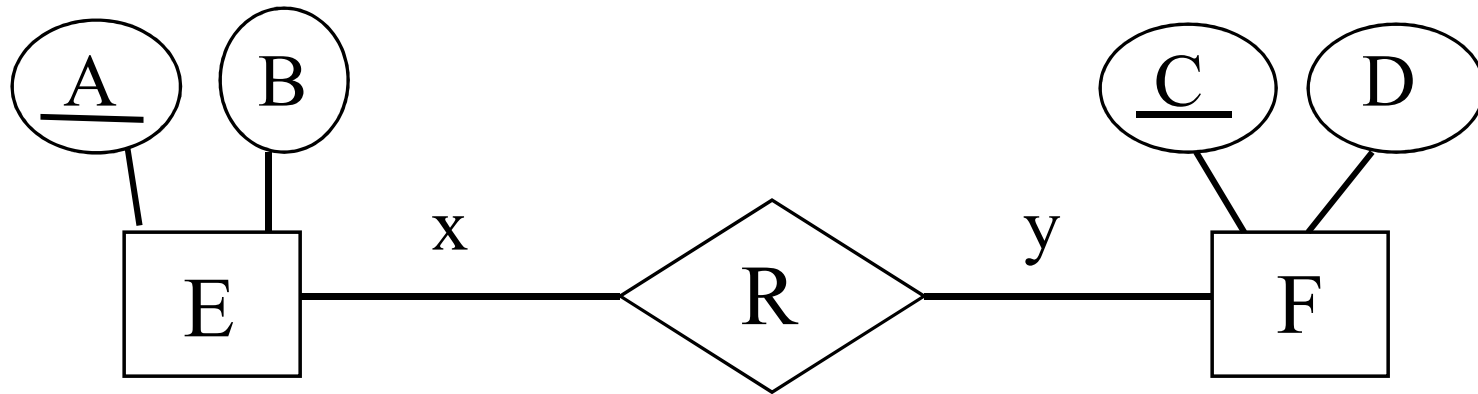
==> Depts(**Name**, Location)

Managers(**Emp#**, Name, Age, **Dept_name**)

==> Depts(**Name**, Location, **Manager_Emp#**)

Managers(**Emp#**, Name, Age)

Transform Binary Relationship (5)

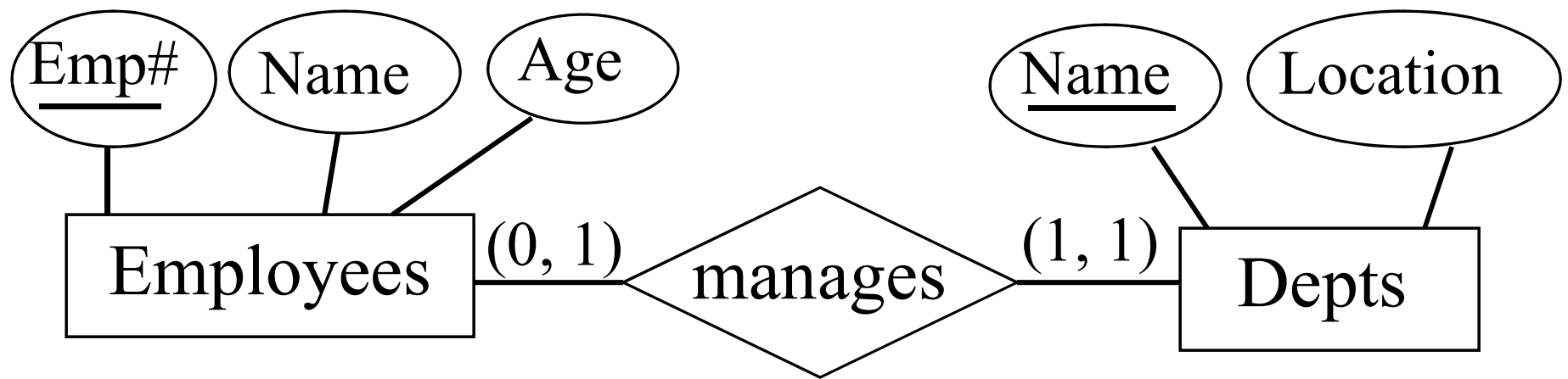


Case 2.2: $x = (0, 1)$ and $y = (1, 1)$

$\implies E(\text{A}, B), F(\text{C}, D, \text{A})$

- The entity set with the total participation is transformed to a relation with a foreign key.

Transform Binary Relationship (6)



==> Depts(**Name**, Location, **Manager_Emp#**)
Employees(**Emp#**, Name, Age)

- Why not let Employees have the foreign key?

Transform Binary Relationship (7)

Transformation Guideline 2: Avoid introducing null values as much as possible. When it is necessary to introduce null values, introduce as few null values as possible.

Transform Binary Relationship (5)

Case 2.3: $x = (0, 1)$ and $y = (0, 1)$

- Transform the relationship set into a foreign key attribute.
- The foreign key should go to the relation that causes the least number of null values.

Transform Binary Relationship (9)

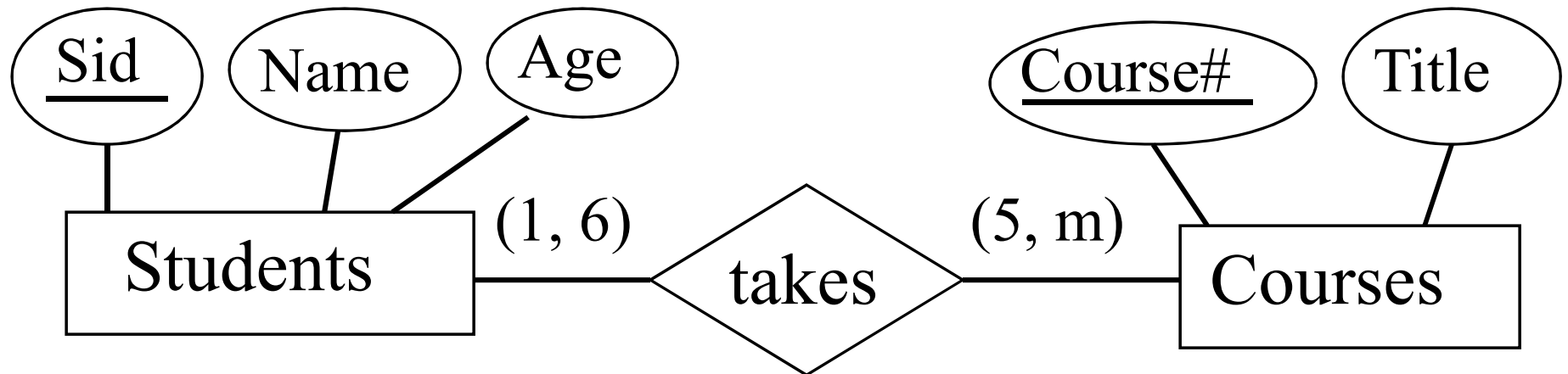
Case 3: many-to-many relationship

Case 3.1: R has no attribute

$\implies E(A, B), F(C, D), R(A, C)$

- Transform the m-to-m relationship to a separate relation.
- R has two foreign keys.
- The key of R consists of the foreign keys.

Transform Binary Relationship (10)



$\Rightarrow$ Students(**Sid**, Name, Age)

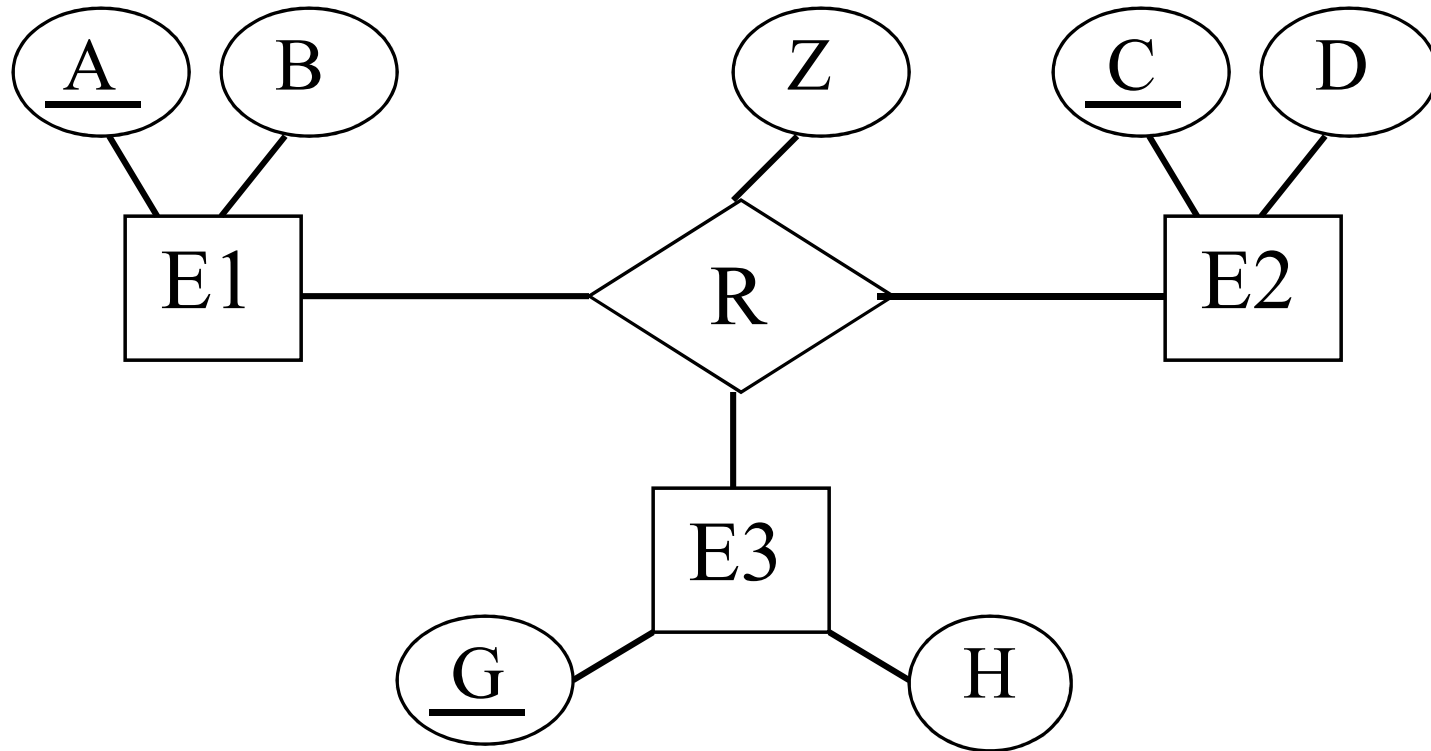
Courses(**Course#**, Title)

Takes(**Sid**, **Course#**)

Case 3.2: R has attribute Z

$\Rightarrow$ E(**A**, B), F(**C**, D), R(**A**, **C**, Z)

Transform Ternary Relationship

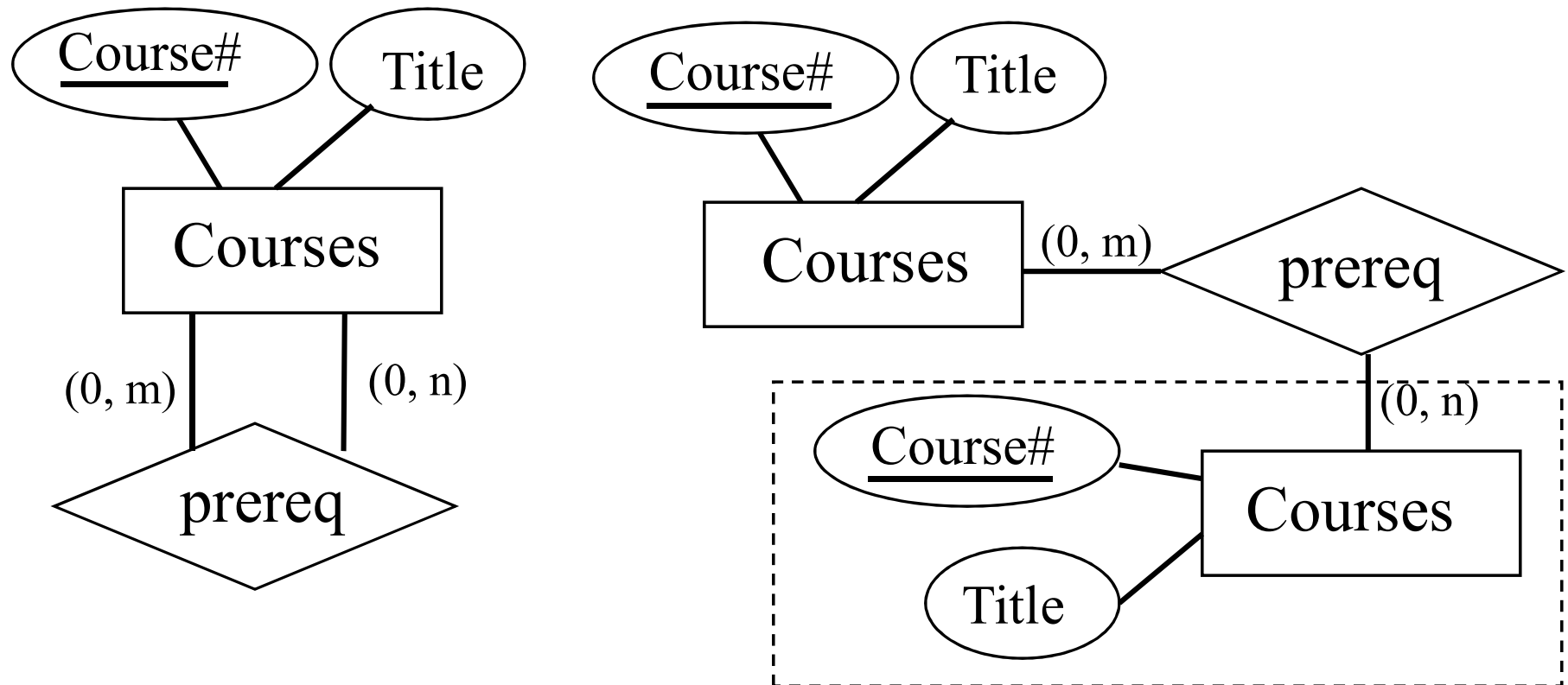


$\Rightarrow E1(\textcolor{brown}{A}, B), E2(\textcolor{brown}{C}, D), E3(\textcolor{brown}{G}, H),$
 $R(\textcolor{teal}{A}, \textcolor{teal}{C}, \textcolor{teal}{G}, Z)$

Transform Unary Relationship (1)

- Create a **shadow** entity set and transform the unary relationship into a binary relationship.
- Apply the rules for transforming binary relationships.
- After the transformation, remove one redundant relation, or if there is no redundant relation, remove the relation with fewer attributes.

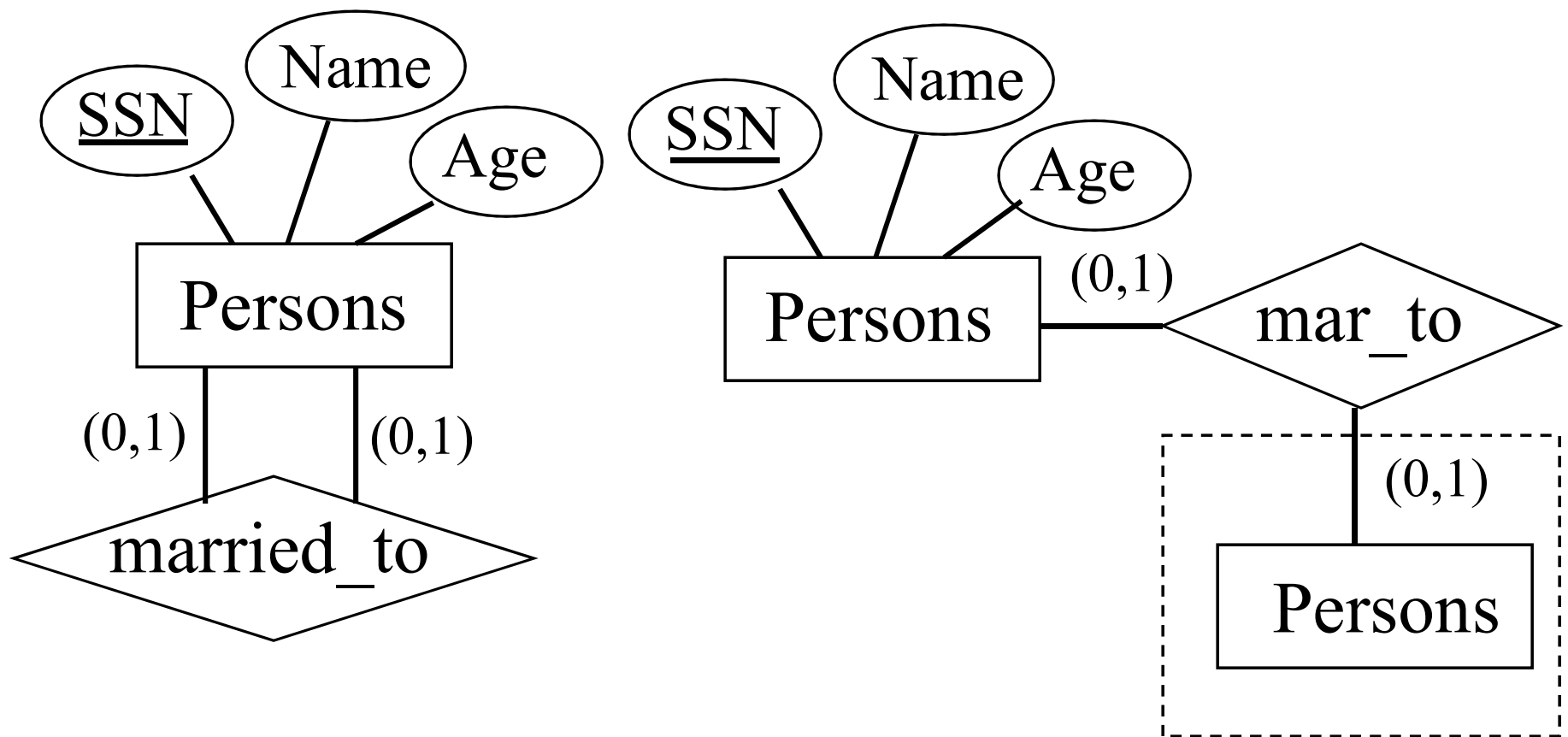
Transform Unary Relationship (2)



Courses(**Course#**, Title)

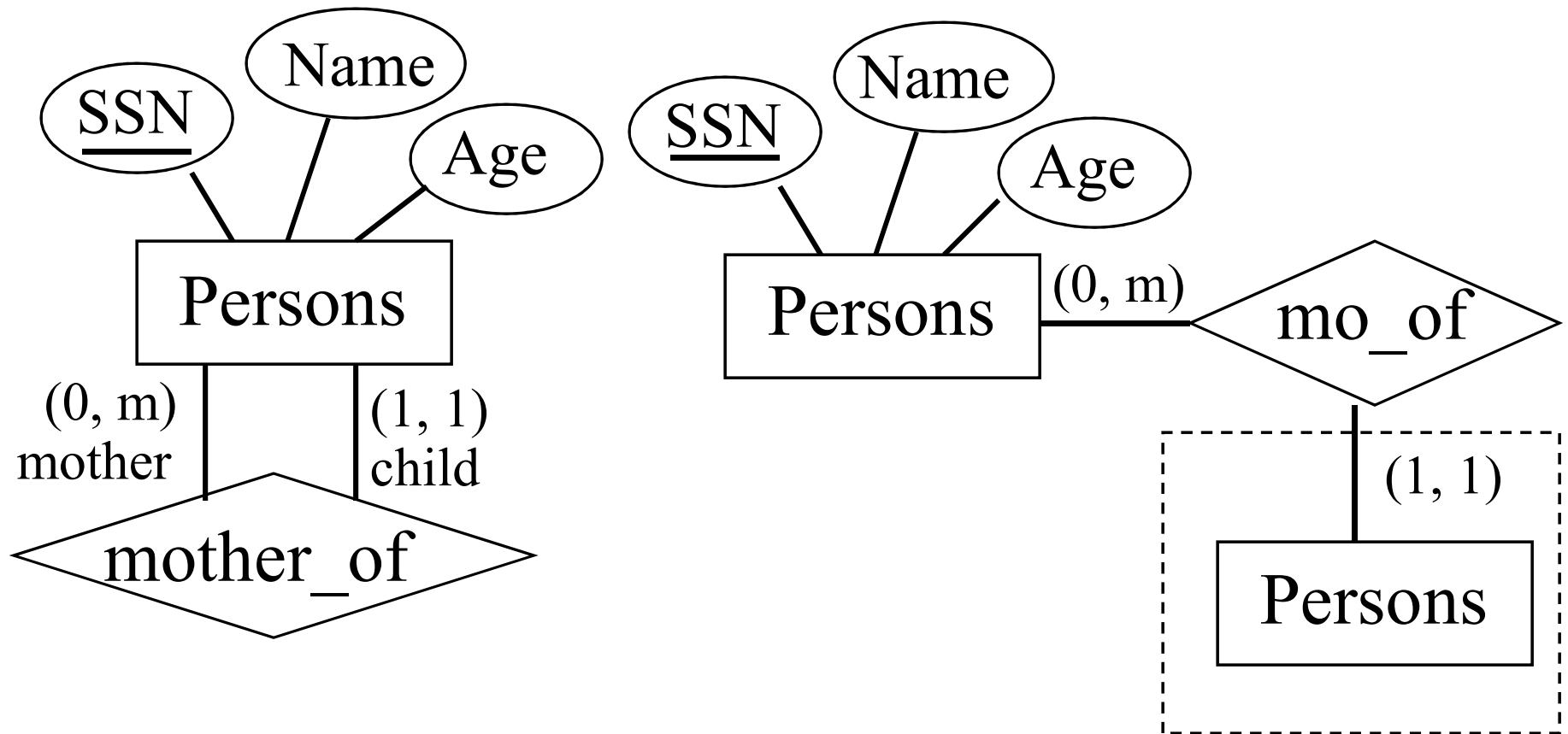
Prereq(**Course#**, Prereq_Course#)

Transform Unary Relationship (3)



Persons(**SSN**, Name, Age, Spouse_SSN)

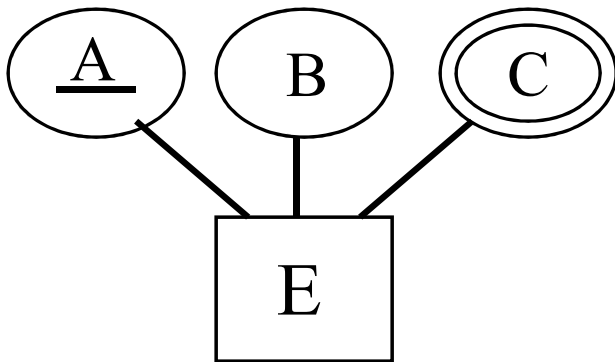
Transform Unary Relationship (4)



Persons(**SSN**, Name, Age, Mother_SSN)

Transform Multi-valued Attribute (1)

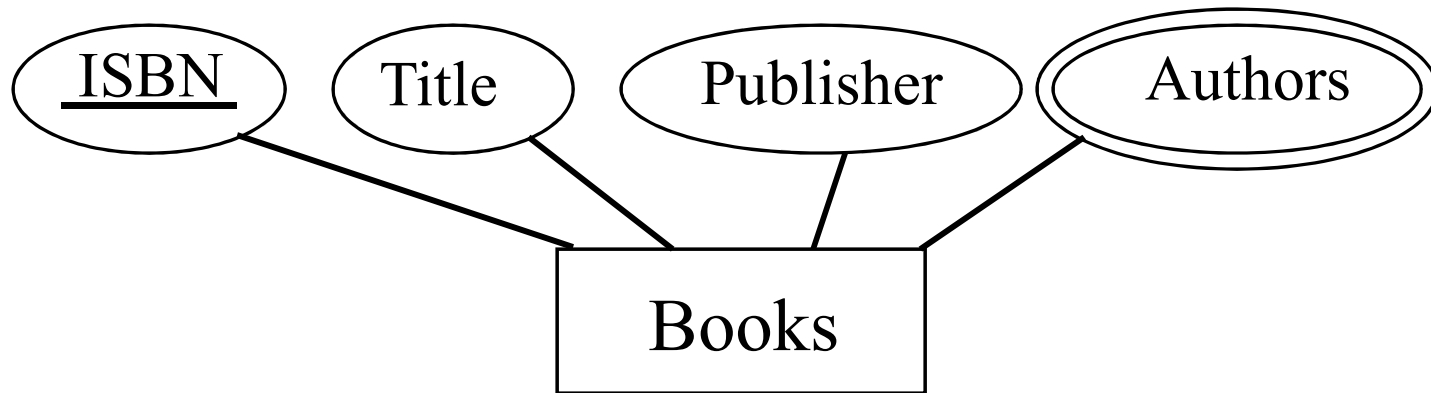
- Create a separate relation for each multi-valued attribute.



$E(\textcolor{brown}{A}, B), E_C(\textcolor{brown}{A}, \textcolor{brown}{C})$

- $E_C.A$ should be defined to be a foreign key referencing $E.A$

Transform Multi-valued Attribute (2)

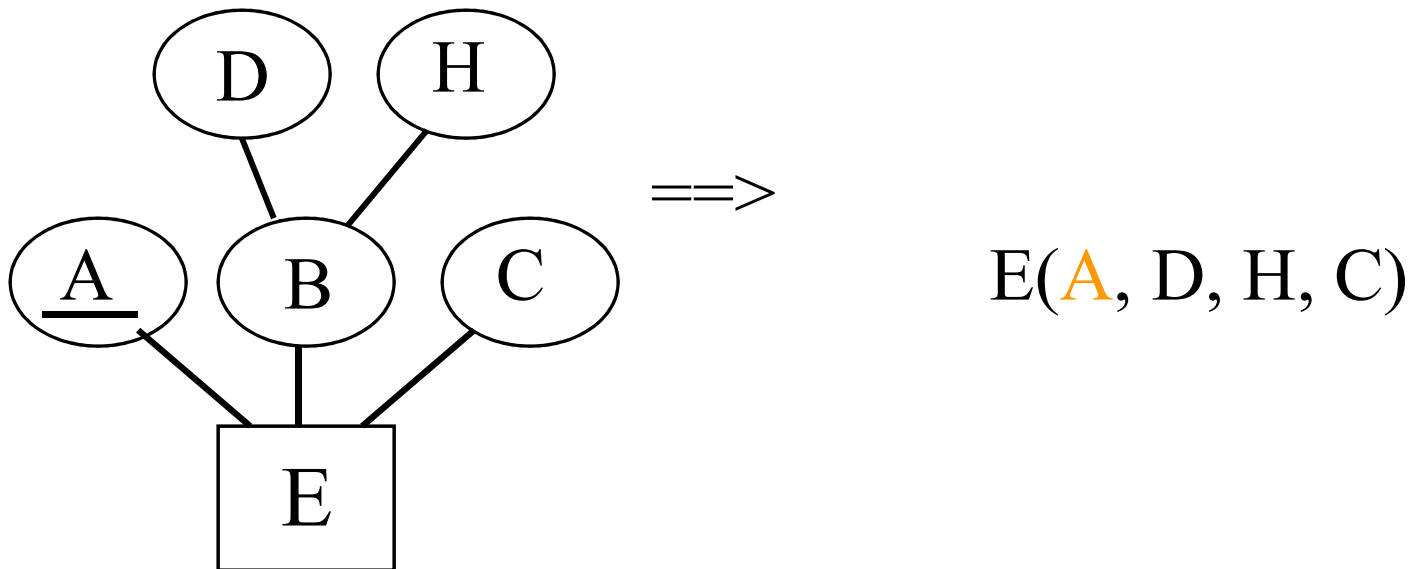


==> Books (ISBN, Title, Publisher)
Book_Authors (ISBN, Author)

- Define Book_Authors.ISBN as a foreign key referencing Books.ISBN

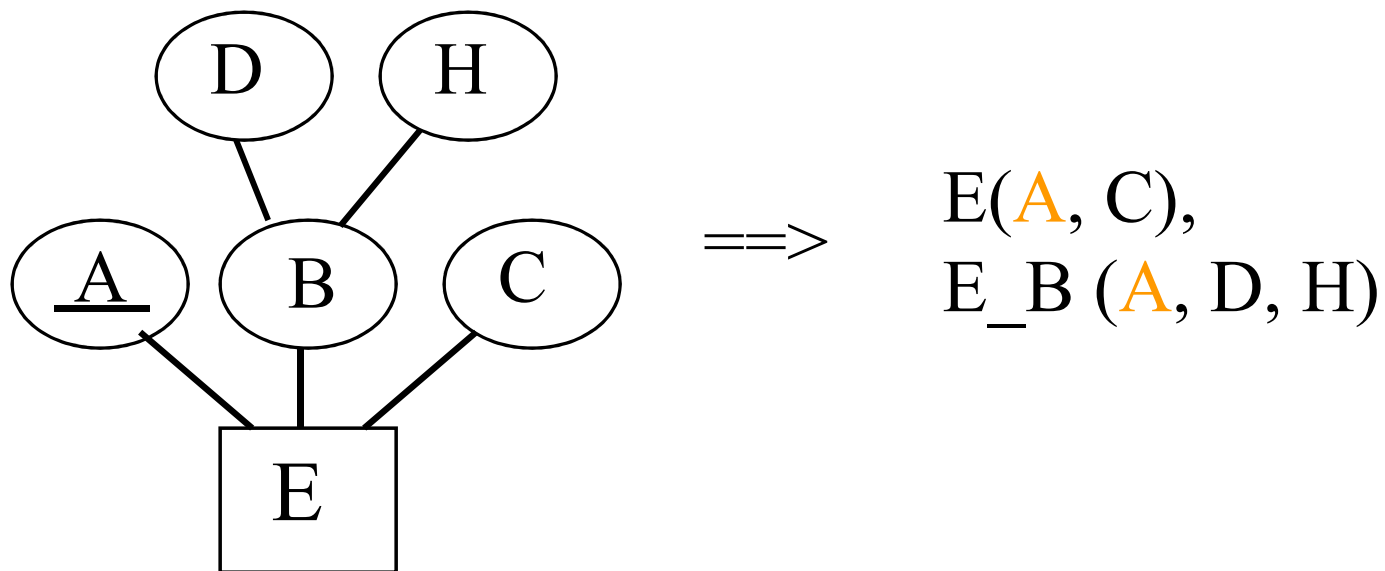
Transform Composite Attribute (1)

Method 1: Use only simple attributes and ignore the composite attribute.



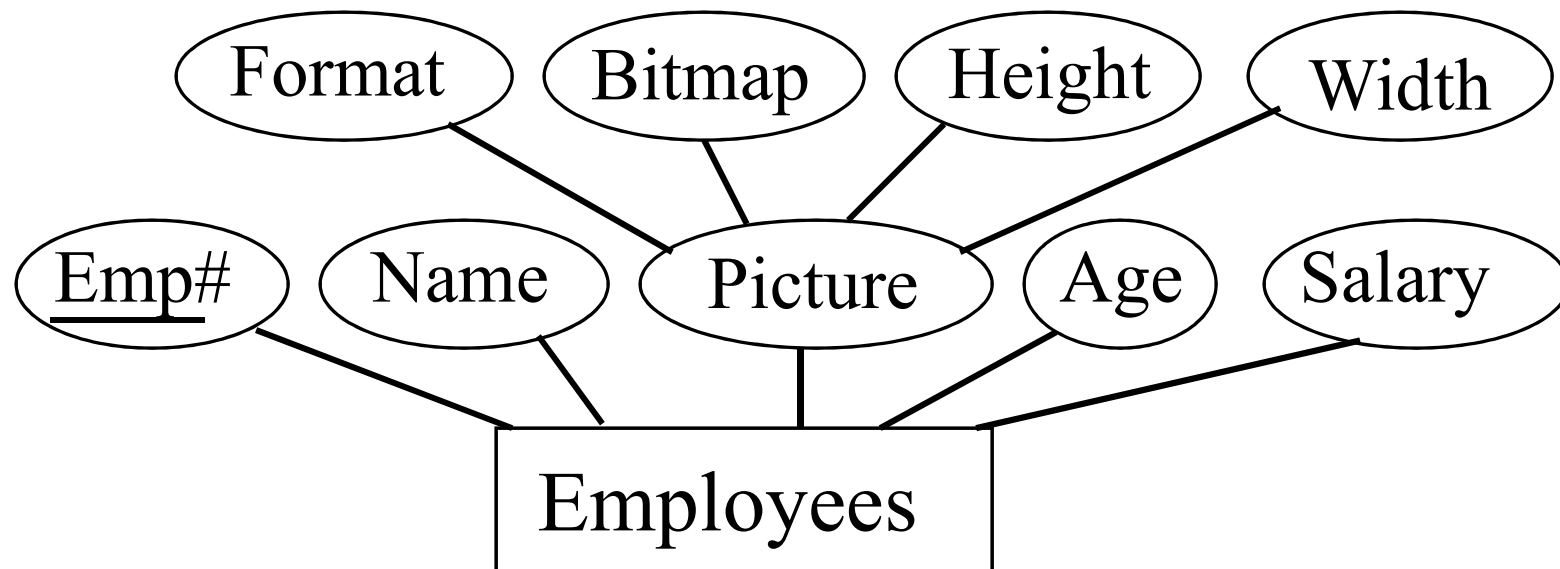
Transform Composite Attribute (2)

Method 2: Transform the composite attribute to a separate relation.



Transform Composite Attribute (3)

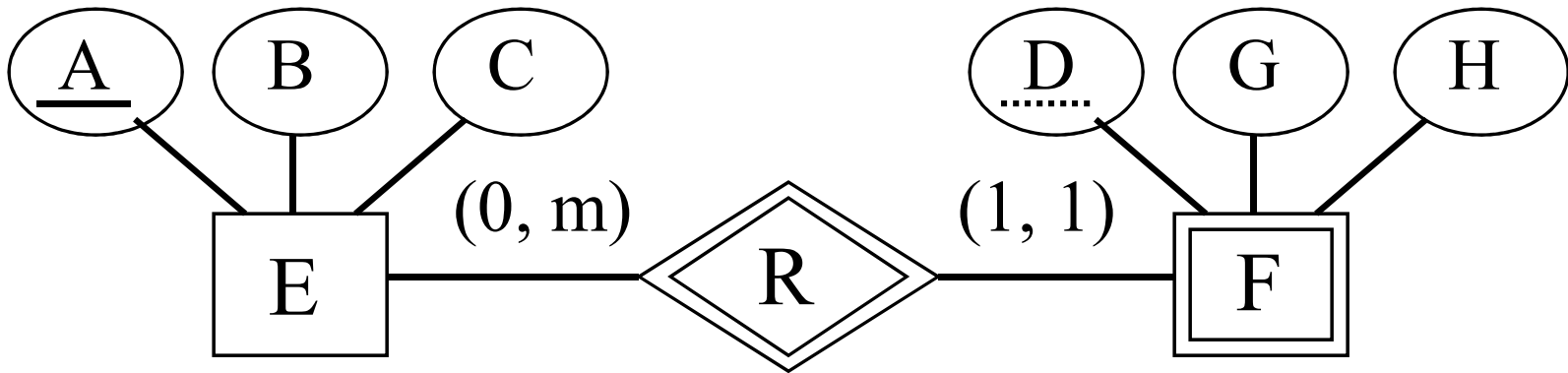
An Example using method 2:



Employees (**Emp#**, Name, Age, Salary)

Emp_Pic (**Emp#**, Bitmap, Format, Height, Width)

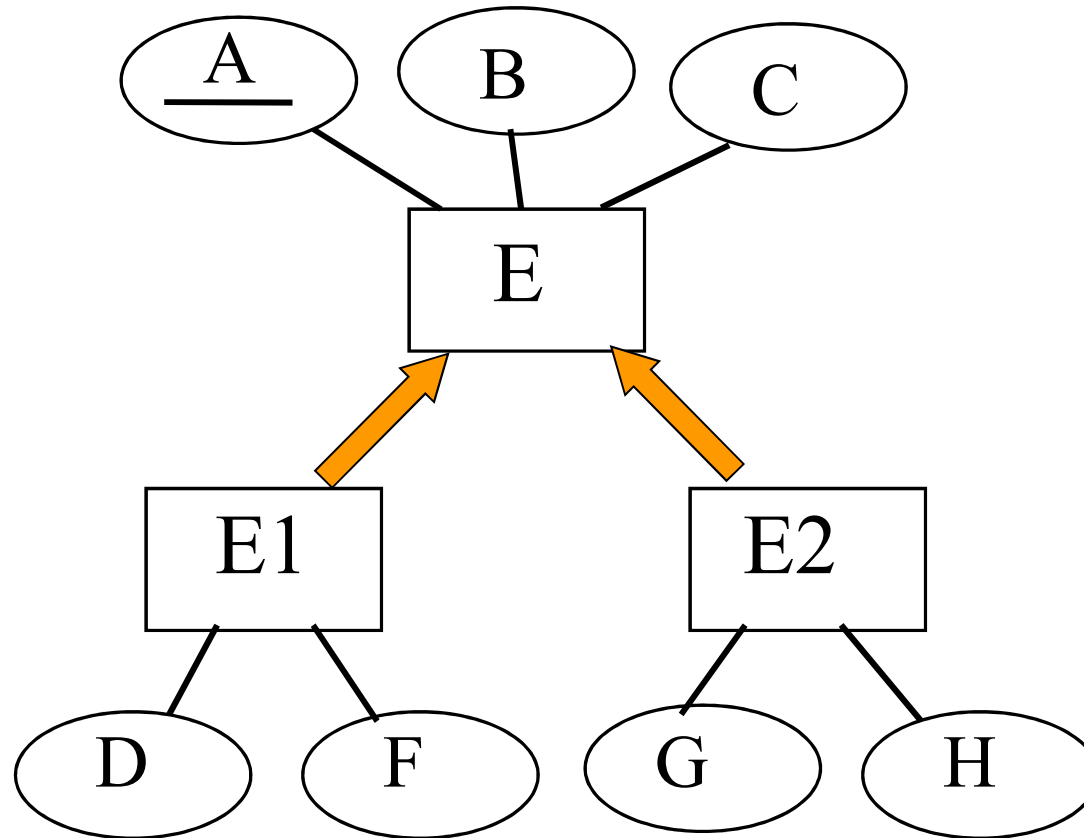
Transform Weak Entity Set



$\Rightarrow E(A, B, C), F(A, D, G, H)$

- The key of F consists of the key of E and the partial key of F .
- $F.A$ is a foreign key referencing $E.A$

Transform IS_A Hierarchy (1)



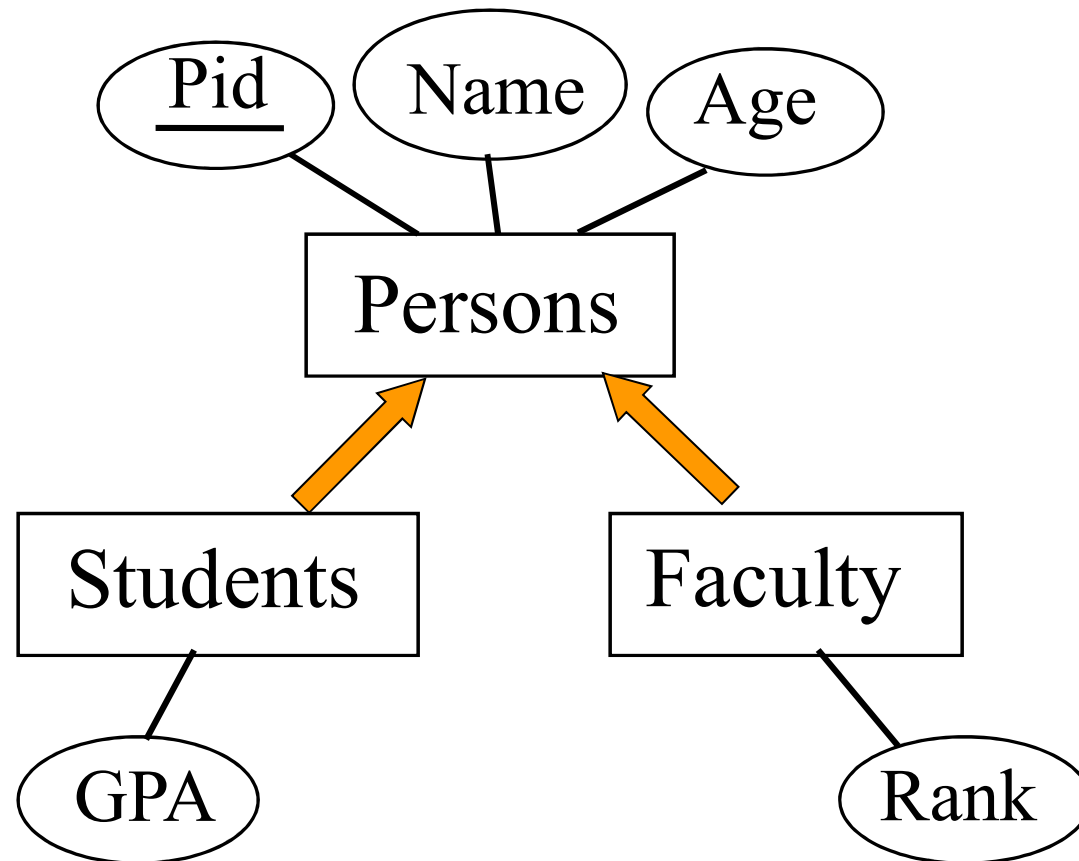
Transform IS_A Hierarchy (2)

Method 1: For general case

$$\implies E(A, B, C), E1(A, D, F), \\ E2(A, G, H)$$

- Only the key is explicitly inherited from the super entity set.
- A tuple in E either corresponds to an entity in E or an entity in a sub entity set.
- E1.A and E2.A are defined to be foreign keys referencing E.A.

Transform IS_A Hierarchy (5)



Transform IS_A Hierarchy (6)

Real world information:

	Pid	Name	Age	GPA	Rank
stud:	123456789	John	27	3.5	
facul:	234567891	Bill	43		Prof.
staff:	345678912	Mary	37		

Transform IS_A Hierarchy (7)

Method 1:

Persons

Pid	Name	Age
123456789	John	27
234567891	Bill	43
345678912	Mary	37

Students

Pid	GPA
123456789	3.5

Faculty

Pid	Rank
234567891	Prof.

Transform IS_A Hierarchy (3)

Method 2: When the union of the sub entity sets contains the same set of entities as the super entity set.

$\Rightarrow E1(A, D, F, B, C), E2(A, G, H, B, C)$

- All attributes are explicitly inherited from the super entity set.
- In principle, this method could also be used when the union of the sub entity sets is not a superset of the super entity set. In this case:

$\Rightarrow E(A, B, C), E1(A, D, F, B, C),$
 $E2(A, G, H, B, C)$

Transform IS_A Hierarchy (8)

Method 2:

Persons

Pid	Name	Age
345678912	Mary	37

Students

Pid	Name	Age	GPA
123456789	John	27	3.5

Faculty

Pid	Name	Age	Rank
234567891	Bill	43	Prof.

Transform IS_A Hierarchy (4)

Method 3: When the sub entity sets are disjoint based on the values of an (implicit) attribute K.

$\Rightarrow E(A, B, C, D, F, G, H, K)$

- K has the same value for entities from the same entity set but different values for entities from different entity sets.

E.g.: Super entity set: Employees

Sub entity sets: Programmers, Analysts, ...

K: Position

- For entities from a sub entity set, fill missing attributed with null values.

Transform IS_A Hierarchy (9)

Method 3:

Persons

Pid	Name	Age	GPA	Rank	Type
123456789	John	27	3.5		Student
234567891	Bill	43		Prof.	Faculty
345678912	Mary	37			Staff